

内核源码阅读与分析报告



姓 名: 马英哲

学 号: 2024090912018

电子科技大学

2026 年 6 月 14 日

摘要

本报告以 Linux 6.18.15 内核源码为分析对象,对进程管理、内存管理、文件系统、进程间通信等核心子系统的设计与实现进行了系统性源码阅读与分析。报告涵盖六个主要部分:(1) 进程组织结构和系统调用实现;(2) 进程调度算法与 x86 上下文切换机制;(3) 进程间通信机制;(4) 物理内存管理与分配;(5) 虚拟存储管理与缺页换页;(6) 虚拟文件系统与 ext2 文件系统实现。每个部分均结合具体源码路径与关键数据结构进行深度解析,并在每章末尾辅以对应的操作系统课程实验验证。

第 2 部分(进程调度与 x86 切换)为本报告重点,对 `__schedule()` 调度主循环、CFS/EEVDF 公平调度算法、`context_switch()` 上下文切换、`switch_to` 宏至 `__switch_to_asm` 的寄存器级切换流程进行了逐层剖析。

关键词: Linux 6.18.15; 进程调度; x86 上下文切换; 内存管理; 虚拟文件系统; ext2; 进程间通信; 内核源码阅读

目录

1 进程组织结构和进程管理系统调用实现	4
1.1 进程控制块— <code>task_struct</code>	4
1.2 进程状态机	5
1.3 进程创建— <code>fork / clone</code> 调用链	5
1.4 系统调用实现	6
1.4.1 系统调用注册	6
1.4.2 系统调用入口— <code>entry_SYSCALL_64</code>	7
1.5 实验验证	7
1.5.1 实验 2: 系统调用添加	7
1.5.2 实验 3: 哲学家进餐多线程实现	7
2 进程调度和 x86 架构下切换机制实现	8
2.1 调度器总体架构	8
2.2 核心调度入口— <code>schedule()</code> 与 <code>__schedule()</code>	9
2.3 <code>pick_next_task()</code> —任务选择逻辑	10
2.4 CFS / EEVDF 公平调度分析	11
2.4.1 从 CFS 到 EEVDF 的演进	11
2.4.2 <code>enqueue_task_fair / dequeue_task_fair</code> —入队出队	11
2.4.3 <code>pick_task_fair()</code> → <code>__pick_eevdf()</code>	12
2.4.4 时钟中断与抢占检查— <code>scheduler_tick()</code>	13
2.5 上下文切换: <code>context_switch()</code>	13
2.5.1 地址空间切换— <code>switch_mm_irqs_off()</code>	13
2.5.2 执行现场切换— <code>switch_to</code> 宏 → <code>__switch_to_asm</code>	14
2.5.3 <code>__switch_to_asm</code> 寄存器状态逐帧分析	15
2.6 完整调度调用链	16
2.7 结构体关系	16
2.8 调度类实战: <code>SCHED_MFQ</code> 完整实现分析	16

2.8.1	MFQ 算法设计	16
2.8.2	内核数据结构嵌入	20
2.8.3	调度类注册与集成	21
2.8.4	调度器初始化	21
2.8.5	完整回调函数逐行分析	22
2.8.6	完整调用链示例：一次抢占切换的逐层追踪	26
2.8.7	用户态调用	26
2.8.8	调度器接口契约：从四个典型错误理解内核设计意图	28
2.9	实验验证	28
3	进程间通信机制	29
3.1	System V IPC 概述	29
3.2	信号量 (Semaphore)	29
3.2.1	数据结构	29
3.2.2	semop() —P/V 操作调用链	29
3.2.3	从 Dijkstra PV 到 Linux semop：三层映射	30
3.3	共享内存 (Shared Memory)	31
3.4	信号 (Signal)	31
3.5	管道 (Pipe)	32
3.6	实验验证	32
3.6.1	实验 4：生产者-消费者多进程实现	32
3.6.2	实验 8：信号量死锁检测与解除	34
4	物理内存管理和分配机制	34
4.1	NUMA 与内存节点	34
4.2	伙伴系统 (Buddy System)	35
4.2.1	核心数据结构	35
4.2.2	分配路径详解	35
4.2.3	释放路径详解	35
4.3	Slub 分配器	36
4.4	实验验证	37
4.4.1	实验 10：内存管理分析与验证（物理内存部分）	37
5	虚拟存储管理和换页机制	37
5.1	虚拟地址空间管理	37
5.2	vm_area_struct (VMA)	37
5.3	mmap 系统调用与惰性分配	38
5.4	缺页异常处理	38
5.5	Copy-on-Write (COW) 深入分析	39
5.6	实验验证	41
5.6.1	实验 10：内存管理分析与验证（虚拟内存部分）	41
5.6.2	实验 5：目录递归拷贝的单/多进程实现	41

6 虚拟文件系统和 ext2 文件系统实现	42
6.1 VFS 虚拟文件系统概述	42
6.2 路径名解析—link_path_walk()	42
6.3 文件操作—read_iter / write_iter	44
6.4 ext2 文件系统实现	44
6.4.1 磁盘布局	45
6.4.2 ext2_inode 与数据块寻址	45
6.4.3 ext2 文件操作	45
6.5 实验验证	46
6.5.1 实验 9: ext2m 加密文件系统实现	46
6.5.2 实验 7: Linux 设备文件与驱动程序	47
7 总结	47
7.1 跨子系统设计模式	48

1 进程组织结构和进程管理系统调用实现

1.1 进程控制块—task_struct

Linux 内核使用 struct task_struct (定义于 include/linux/sched.h) 作为进程描述符, 是内核感知进程存在的唯一依据 [8, 9]。该结构体是 Linux 内核中最大的数据结构之一 (在 6.18.15 中约 700+ 行), 包含了进程调度、内存管理、文件系统、信号处理等全部元信息。

核心字段分类如下:

- **进程标识与状态:** pid_t pid (通过 task_pid_nr() 获取数值)、unsigned int __state (进程状态位掩码)、int exit_code (退出码)。
- **调度信息:** int prio (动态优先级)、int static_prio (静态优先级 100–139)、unsigned int policy (调度策略)、struct sched_entity se (CFS 调度实体)、struct sched_rt_entity rt (RT 调度实体, 含 time_slice 字段)、struct sched_dl_entity dl (DL 调度实体)。
- **内存管理:** struct mm_struct *mm (用户空间内存描述符)、struct mm_struct *active_mm (内核线程借用的活跃内存描述符)。
- **文件系统:** struct fs_struct *fs (文件系统上下文, 含 root/pwd)、struct files_struct *files (打开文件描述符表)。
- **线程信息:** struct thread_struct thread (架构相关寄存器状态, 含 sp 栈指针)。

```

1 // include/linux/sched.h
2 struct task_struct {
3     unsigned int    __state;           // 进程状态
4     void            *stack;           // 内核栈指针
5     int             prio;             // 动态优先级
6     int             static_prio;      // 静态优先级
7     int             normal_prio;      // 归一化优先级
8     unsigned int    rt_priority;      // 实时优先级(1-99)
9     unsigned int    policy;           // 调度策略
10    struct sched_entity se;           // CFS 调度实体

```

```

11  struct sched_rt_entity  rt;           // RT 调度实体
12  struct sched_dl_entity  dl;           // DL 调度实体
13  struct mm_struct        *mm;           // 用户地址空间
14  struct mm_struct        *active_mm;    // 活跃地址空间
15  pid_t                   pid;           // 进程 ID
16  struct list_head        tasks;         // 全局进程链表
17  struct thread_struct     thread;        // 架构相关寄存器状态
18  struct files_struct      *files;        // 打开文件表
19  struct fs_struct         *fs;           // 文件系统上下文
20  // ... 总计 700+ 行
21  };

```

Listing 1: task_struct 调度相关核心字段

1.2 进程状态机

Linux 6.18.15 中进程状态通过 `__state` 字段的位掩码表示：

- `TASK_RUNNING` (0x0000)：正在运行或就绪（在运行队列中）。
- `TASK_INTERRUPTIBLE` (0x0001)：可中断睡眠，可被信号唤醒。
- `TASK_UNINTERRUPTIBLE` (0x0002)：不可中断睡眠（ps 中显示为 D 状态），典型场景为等待磁盘 I/O 完成。
- `TASK_STOPPED` (0x0004)：收到 `SIGSTOP` 信号或正在被 `ptrace` 调试。
- `TASK_TRACED` (0x0008)：被 `ptrace` 跟踪（如 `gdb`）。
- `EXIT_DEAD` / `EXIT_ZOMBIE` (0x0010/0x0020)：进程已退出但尚未被父进程回收。

状态转换通过 `set_current_state()` 和 `__set_current_state()` 宏实现（`include/linux/sched/task.h`），内部调用 `WRITE_ONCE(current->__state, value)` 保证编译器不优化状态写入。这里使用 `WRITE_ONCE` 而非普通赋值的原因是：进程状态可能被中断上下文或远程 CPU 并发读取（如调度器的 `try_to_wake_up()` 检查睡眠进程状态），`WRITE_ONCE` 确保写入的原子性和可见性，防止编译器撕裂写入或重排顺序。

状态转换的一个典型场景是进程等待 I/O 完成：进程在 `wait_event()` 宏中调用 `__set_current_state(TASK_UNINTERRUPTIBLE)`，将自身标记为不可中断睡眠，然后调用 `schedule()` 让出 CPU。I/O 完成后，设备驱动程序的中断处理函数调用 `wake_up()` → `__wake_up_common()` → `try_to_wake_up()`，将进程状态从 `TASK_UNINTERRUPTIBLE` 改回 `TASK_RUNNING` 并加入运行队列。整个过程中，进程从“主动睡眠 → CPU 上消失 → 中断唤醒 → 重新就绪”的完整生命周期仅在若干状态位翻转之间完成，体现了 Unix 进程模型的优雅性。

进程状态变迁关系如图 1 所示。

1.3 进程创建—fork / clone 调用链

进程创建的唯一入口是 `kernel/fork.c` 中的 `kernel_clone()`，它是 `fork()`、`vfork()`、`clone()` 三个系统调用的共同底层实现 [15]。

```

1  // kernel/fork.c
2  pid_t kernel_clone(struct kernel_clone_args *args) {
3      // 1. 复制进程描述符——这是最核心的步骤

```

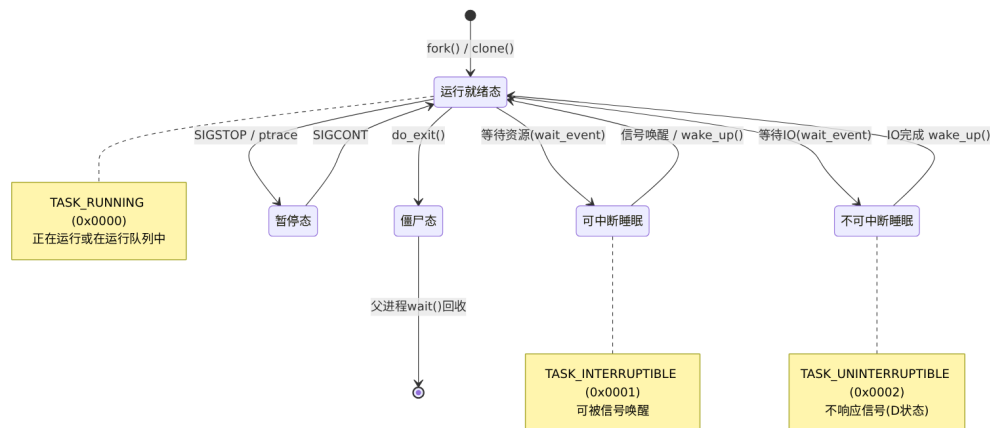


图 1: Linux 进程状态机

```

4  p = copy_process(NULL, trace, NUMA_NO_NODE, args);
5  // 2. 唤醒新进程加入调度队列
6  wake_up_new_task(p);
7  // 3. 返回子进程 PID
8  return p->pid;
9  }

```

Listing 2: kernel_clone 核心流程

`copy_process()` (`kernel/fork.c`) 是进程创建的核心，调用链如下：

步骤 1 —`dup_task_struct()`: 通过 `alloc_task_struct_node()` 分配新的 `task_struct` 和内核栈（通常为 `THREAD_SIZE = 16KB`，包含 `thread_info` 和内核栈），将父进程的 `task_struct` 整体复制到子进程，然后清零子进程的特有字段（如 `pid`, `thread_pid`）。

步骤 2 —`sched_fork()`: 初始化新进程的调度实体。将 `se.vruntime` 设置为父进程的 `vruntime`（保证公平性），设置 `se.on_rq = 0`（尚未入队），初始化 `se.load` 权重为 `NICE_0_LOAD` (1024)。对于 `EEVDF`，还需初始化 `deadline`, `vlag` 等字段。

步骤 3 —`copy_mm()`: 根据 `clone flags` 决定共享还是复制地址空间。对于 `fork()`（非 `CLONE_VM`），调用 `dup_mm()` → `dup_mmap()` 复制父进程的所有 `VMA`，同时将父子进程的页表项均标记为只读（`COW` 准备）。对于 `clone(CLONE_VM)`（创建线程），直接共享 `mm_struct`。

步骤 4 —`copy_thread()`: 架构相关初始化（`arch/x86/kernel/process_64.c`）。设置 `thread.sp` 指向子进程内核栈上预留的 `pt_regs` 区域底部，设置 `thread.ip` 为 `ret_from_fork` 汇编入口。

步骤 5 —唤醒: `wake_up_new_task()` → `activate_task()` → `enqueue_task()`，将子进程加入 CFS 红黑树，若子进程优先级高于当前进程则设置 `TIF_NEED_RESCHED` 标志。

1.4 系统调用实现

1.4.1 系统调用注册

在 Linux 6.x 中，添加系统调用仅需修改三个文件 [8]：

1. `arch/x86/entry/syscalls/syscall_64.tbl`: 注册系统调用号、ABI 类型、函数名。
2. `kernel/sys.c` 或其他子系统文件: 使用 `SYSCALL_DEFINE()` 宏实现系统调用函数体。

3. `include/uapi/asm-generic/unistd.h`: 更新 `__NR_syscalls` 的值。

1.4.2 系统调用入口—`entry_SYSCALL_64`

`x86_64` 下系统调用通过 `syscall` 指令进入内核 (`arch/x86/entry/entry_64.S`) [17], 硬件自动将 `RIP` 保存到 `RCX`、`RFLAGS` 保存到 `R11`, 从 `IA32_LSTAR` MSR 加载内核入口 `RIP` (`entry_SYSCALL_64`), 切换到 Ring 0 并使用内核栈。

`entry_SYSCALL_64` 汇编代码经过 `do_syscall_64()` (`arch/x86/entry/common.c`) 根据 `RAX` 中的系统调用号查找 `sys_call_table` 数组, 调用对应处理函数:

```
1 // arch/x86/entry/common.c
2 __visible noinstr void do_syscall_64(struct pt_regs *regs, int nr) {
3     if (likely(nr < NR_syscalls)) {
4         regs->ax = sys_call_table[nr](regs); // 返回结果写入 RAX
5     }
6 }
```

Listing 3: `do_syscall_64` —系统调用分发

1.5 实验验证

1.5.1 实验 2: 系统调用添加

本实验完成了向 Linux 6.18.15 内核添加自定义系统调用 `sys_my_add` 的全流程验证。内核侧修改三个文件: (1) `arch/x86/entry/syscalls/syscall_64.tbl` 注册 470 `common my_add sys_my_add`; (2) `kernel/sys.c` 中使用 `SYSCALL_DEFINE2(my_add, int a, int b)` 实现加法并添加参数合法性检查——负值输入返回 `-EINVAL`, 同时输出 `printk(KERN_INFO)` 内核日志; (3) `include/uapi/asm-generic/unistd.h` 将 `__NR_syscalls` 从 470 递增至 471。

编译采用增量方式 (`make -j$ (nproc) bzImage`), 验证方式为用户态程序通过 `syscall(__NR_my_add, 10, 20)` 绕过 `glibc` 包装直接调用内核。返回值确认为 30 (10+20), `dmesg` 输出 "This is a new syscall: my_add(10, 20) called"。内核编译系统自动从 `.tbl` 文件生成 `unistd_64.h` (含 `__NR_my_add 470`) 和 `syscalls_64.h` (含 `__SYSCALL(470, sys_my_add)`), 验证了 Linux 6.x 系统调用表的自动生成机制。

该实验直接印证了 §1.4 所述的系统调用全路径: 用户态 `syscall` 指令 → 硬件切换到 Ring 0 → `entry_SYSCALL_64` → `do_syscall_64()` 根据 `RAX` 索引 `sys_call_table[470]` → 调用 `__x64_sys_my_add()` → `sysretq` 返回用户态。

1.5.2 实验 3: 哲学家进餐多线程实现

本实验使用 POSIX 线程库实现了经典哲学家进餐问题的两个版本。死锁版本中五哲学家均遵循“先左叉后右叉”的对称策略, 预防版本引入**偏序资源分配**: 偶数号哲学家改为“先右叉后左叉”, 从而打破循环等待环路。

线程同步通过 `pthread_mutex_t` 实现, 五把互斥锁对应五把叉子。死锁检测由独立的观察者线程完成——该线程睡眠 5 秒后检查是否有哲学家仍处于 `HUNGRY` 状态且未达到预定进餐次数。测试采用自动化脚本连续运行 30 次预防版本, 统计成功率。

实验结果: 死锁版本在启动后数秒内稳定陷入死锁——所有五个哲学家均持有左叉等待右叉, 形成完美的循环等待拓扑。预防版本 30 次运行全部成功 (100% 成功率), 无一次死锁发生。该实验验证了 Coffman 死锁四条件中“循环等待”的破坏策略, 从用户态层面印证了内核中 `futex` 系统调

用对 `pthread_mutex_lock()` 的支撑——未竞争时通过原子操作在用户态完成加锁，竞争时才进入内核调用 `futex(FUTEX_WAIT)` 进入睡眠。

2 进程调度和 x86 架构下切换机制实现

本章为本报告重点章节，对 Linux 6.18.15 的进程调度核心逻辑与 x86_64 架构下的上下文切换机制进行逐层深入的源码级分析。

2.1 调度器总体架构

Linux 6.18.15 内核采用**调度类层次结构**（Scheduling Class Hierarchy）[8, 9, 2]，每个调度类实现一组标准的 `sched_class` 回调接口。调度类按优先级从高到低排列如表 1 所示。

表 1: Linux 6.18.15 调度类层次

优先级	调度类	策略	说明
0 (最高)	<code>stop_sched_class</code>	—	CPU 热插拔、任务迁移等内核关键操作
1	<code>dl_sched_class</code>	<code>SCHED_DEADLINE</code>	限期调度，基于 EDF+CBS 算法
2	<code>rt_sched_class</code>	<code>SCHED_FIFO</code> / <code>SCHED_RR</code>	实时调度，优先级 1-99
3	<code>fair_sched_class</code>	<code>SCHED_NORMAL</code> / <code>BATCH</code>	CFS/EEVDF 公平调度，优先级 100-139
4 (最低)	<code>idle_sched_class</code>	<code>SCHED_IDLE</code>	每 CPU 的 idle 任务

调度类的链接顺序由 `include/asm-generic/vmlinux.lds.h` 中的链接脚本控制，保证遍历从最高优先级开始。`DEFINE_SCHED_CLASS` 宏（`kernel/sched/sched.h`）定义每个调度类实例，包含约 20 个回调函数指针：

```
1 struct sched_class {
2     unsigned int idx;
3     void (*enqueue_task)(struct rq *rq, struct task_struct *p, int flags);
4     void (*dequeue_task)(struct rq *rq, struct task_struct *p, int flags);
5     void (*yield_task)(struct rq *rq);
6     void (*wakeup_preempt)(struct rq *rq, struct task_struct *p, int flags);
7     struct task_struct *(*pick_task)(struct rq *rq);
8     struct task_struct *(*pick_next_task)(struct rq *rq);
9     void (*put_prev_task)(struct rq *rq, struct task_struct *p);
10    void (*set_next_task)(struct rq *rq, struct task_struct *p, bool first);
11    void (*task_tick)(struct rq *rq, struct task_struct *p, int queued);
12    void (*task_fork)(struct task_struct *p);
13    void (*task_dead)(struct task_struct *p);
14    void (*switched_from)(struct rq *rq, struct task_struct *p);
15    void (*switched_to)(struct rq *rq, struct task_struct *p);
16    void (*update_curr)(struct rq *rq);
17 };
```

Listing 4: `sched_class` 核心回调（`kernel/sched/sched.h`）

2.2 核心调度入口—schedule() 与 __schedule()

schedule() (kernel/sched/core.c) 在以下场景被调用 [24]: 进程主动调用 sched_yield() 或 schedule()、等待资源时调用 schedule_timeout() / wait_event()、从中断/异常返回前检测到 TIF_NEED_RESCHED 标志。

```
1 void __sched schedule(void) {
2     struct task_struct *tsk = current;
3     sched_submit_work(tsk);
4     __schedule(SM_NONE);
5     sched_update_worker(tsk);
6 }
```

Listing 5: schedule() 入口 (kernel/sched/core.c)

__schedule() 是真正的调度决策函数，完成**选任务** → **切上下文**全流程。理解该函数的关键在于认识到：调用 schedule() 的进程将自己视为“prev”——它主动交出 CPU，期望调度器选择另一个进程 (“next”) 运行。

分裂返回是该函数最反直觉的语义。以下从 prev 和 next 两个视角展示一次完整的调度-切换-返回全时序：

表 2: __schedule() 分裂返回：prev 与 next 的双视角时序

步骤	执行者	prev 视角	next 视角
1	prev	schedule() → __schedule() 入口, prev 将自己设为当前任务	next 此时可能在就绪队列中等待, 或在某个等待队列中睡眠
2	prev	pick_next_task() 选中 next	(无感知)
3	prev	context_switch() 执行 switch_mm_irqs_off() 切换地址空间	(无感知)
4	prev	switch_to(prev, next, prev) → __switch_to_asm: push r15..rbp 保存到 prev 内核栈, mov %rsp, prev->thread.sp	(无感知)
►	RSP 切换点	mov next->thread.sp, %rsp: 从此 RSP 指向 next 的内核栈	pop r15..rbp: 恢复上次自己被切出时的寄存器, jmp __switch_to
5	next	prev 的指令流在此冻结, CPU 继续执行但“prev 不知道”	__switch_to() 收尾: 更新 current_task、切换 FPU/FSGSBASE、返回 prev 指针
6	next	(冻结中)	context_switch() 返回 → __schedule() 返回 → schedule() 返回。next 继续执行自己的业务逻辑
7	将来的调度	某次调度器再次选中 prev, 从 prev->thread.sp 恢复 RSP, pop 寄存器, jmp __switch_to → prev 从 context_switch 返回	(无感知)
8	prev	prev 从 __schedule() 返回, 继续执行——它对步骤 5-7 之间发生的一切毫无记忆	(可能已被切出)

这张表揭示了一个关键事实：**prev 在 __switch_to_asm 的 RSP 切换指令处被“冻结”，而 CPU 从此刻开始执行的是 next 的代码。**prev 内核栈上压着它的 callee-saved 寄存器，prev->thread.sp 指向那个位置。将来 prev 被再次调度时，__switch_to_asm 后半段把寄存器全部弹回，jmp __switch_to 进入 C 函数，然后一路返回到 context_switch → __schedule → schedule。prev 对切换期间的所有事件（中断、其他进程的执行、时钟推进）完全无感知——它的“时间”在 RSP 切换点暂停，在恢复点继续。这就是为什么调度器代码中 schedule() 之后的代码不能假设任何有关“刚才发生了什么”的事情——你所在的 CPU 可能已经运行过数十个其他进程，你的 cache 行可能全部被逐出，你的 TLB 条目可能已被覆盖。这种“分裂返回”语义是理解整个调度器和上下文切换机制的核心。

```

1 static void __sched notrace __schedule(int sched_mode) {
2     struct task_struct *prev, *next;
3     struct rq *rq;
4     cpu = smp_processor_id();
5     rq = cpu_rq(cpu);           // 获取当前CPU运行队列
6     prev = rq->curr;           // 当前运行进程 = 即将变为 prev
7
8     rq->donor = prev;
9     SCHED_WARN_ON(current != prev);
10
11     // 核心：选择下一个要运行的任务
12     pick_again:
13     next = pick_next_task(rq, prev, &rf);
14     if (next == prev) {
15         rq->clock_update_flags &= ~(RQCF_ACT_SKIP|RQCF_REQ_SKIP);
16         goto idle_exit;
17     }
18
19     // 上下文切换：prev → next
20     rq = context_switch(rq, prev, next, &rf);
21     // 返回后，当前 CPU 已在 next 的上下文中执行
22 }

```

Listing 6: __schedule() 核心流程 (kernel/sched/core.c)

2.3 pick_next_task() — 任务选择逻辑

```

1 static struct task_struct *
2 pick_next_task(struct rq *rq, struct task_struct *prev,
3               struct rq_flags *rf) {
4     // 快速路径：若所有就绪任务都是 CFS 普通进程
5     if (likely(prev->sched_class == &fair_sched_class &&
6               rq->nr_running == rq->cfs.h_nr_running)) {
7         p = pick_next_task_fair(rq, prev, rf);
8         if (unlikely(p == RETRY_TASK))
9             goto restart;
10        return p;
11    }
12    return __pick_next_task(rq, prev, rf);
13 }
14
15 static struct task_struct *
16 __pick_next_task(struct rq *rq, struct task_struct *prev,
17                 struct rq_flags *rf) {
18     // 遍历所有调度类：stop → dl → rt → fair → idle
19     for_each_active_class(class) {
20         p = class->pick_next_task(rq);
21         if (p) return p;
22     }
23     BUG(); // idle 类应始终返回 idle 任务
24 }

```

Listing 7: pick_next_task 与 __pick_next_task (kernel/sched/core.c)

for_each_active_class 宏从最高优先级调度类 (stop) 开始遍历，调度类遍历顺序与表 1 一致 [2]。

2.4 CFS / EEVDF 公平调度分析

2.4.1 从 CFS 到 EEVDF 的演进

Linux 6.18.15 的 CFS 调度器已从纯虚拟运行时间 (vruntime) 模型演进为 **EEVDF (Earliest Eligible Virtual Deadline First)** 算法 [3, 21, 26]。关键差异如表 3 所示。

表 3: CFS vs EEVDF 核心差异

维度	CFS (经典)	EEVDF (6.6+)
选择依据	最小 vruntime	最小 virtual deadline
时间片分配	静态计算	动态 request_size
延迟保证	近似	精确 (有界延迟)
实体字段	vruntime	deadline, min_vruntime, vlag

EEVDF 的核心思想：调度实体请求一段运行时间，调度器为其分配一个虚拟截止时间 (virtual deadline)，每次选择 deadline 最早且已 eligible 的实体运行。eligibility 通过比较实体的 vlag 和 min_vruntime 动态判定。

```
1 struct sched_entity {
2     struct load_weight    load;           // 负载权重
3     struct rb_node        run_node;       // 红黑树节点
4     unsigned int          on_rq;         // 是否在运行队列
5     u64                   exec_start;     // 本次运行起始时间
6     u64                   sum_exec_runtime; // 累计运行时间
7     u64                   vruntime;      // 虚拟运行时间
8     u64                   prev_sum_exec_runtime;
9     u64                   deadline;      // EEVDF: 虚拟截止时间
10    u64                   min_vruntime;   // EEVDF: 最小VRuntime边界
11    u64                   vlag;          // EEVDF: 滞后值 (lag)
12    struct sched_entity    *parent;
13    struct cfs_rq          *cfs_rq;      // 所属 CFS 运行队列
14};
```

Listing 8: sched_entity —EEVDF 核心字段 (include/linux/sched.h)

2.4.2 enqueue_task_fair / dequeue_task_fair —入队出队

enqueue_task_fair() (kernel/sched/fair.c) 是任务加入调度队列的核心：

```
1 static void enqueue_task_fair(struct rq *rq, struct task_struct *p, int flags) {
2     struct cfs_rq *cfs_rq = cfs_rq_of(&p->se);
3     // 1. 若实体已在队列，直接返回 (如 migration 场景)
4     if (p->se.on_rq) return;
5     // 2. 更新 vruntime (新唤醒的任务可能需补偿休眠时间)
6     if (!(flags & ENQUEUE_WAKEUP) || (flags & ENQUEUE_MIGRATED))
7         p->se.vruntime += cfs_rq->min_vruntime;
8     // 3. 核心：插入 EEVDF 红黑树
9     enqueue_entity(cfs_rq, &p->se, flags);
10    // 4. 增加 nr_running 计数
11    cfs_rq->h_nr_running++;
12    // 5. 若新任务 deadline 早于当前任务，触发抢占
13}
```

Listing 9: enqueue_task_fair 核心逻辑 (kernel/sched/fair.c)

`enqueue_entity()` 内部调用 `__enqueue_entity()` 将 `se->run_node` 插入 `cfs_rq->tasks_timeline` 红黑树（按 `deadline` 排序），调用 `update_min_vruntime()` 更新 CFS 运行队列的最小虚拟运行时间，若新实体既不在运行中也不是当前实体，则调用 `update_cfs_group()` 更新组调度统计。

`dequeue_task_fair()` 执行反向操作：从红黑树中移除实体，递减 `nr_running`，更新 `min_vruntime`。

2.4.3 `pick_task_fair()` → `__pick_eevdf()`

CFS/EEVDF 的任务选择入口为 `pick_task_fair()`：

```
1 static struct task_struct *pick_task_fair(struct rq *rq) {
2     struct cfs_rq *cfs_rq = &rq->cfs;
3     struct sched_entity *se;
4     if (!cfs_rq->nr_queued) return NULL;
5     // 调用 EEVDF 核心：选择最优调度实体
6     se = pick_next_entity(rq, cfs_rq);
7     return task_of(se); // 从 sched_entity 反向获取 task_struct
8 }
```

Listing 10: `pick_task_fair` (kernel/sched/fair.c)

`pick_next_entity()` 优先检查是否有预设的 `->next` 伙伴进程，否则调用 `__pick_eevdf()` 从红黑树中选择：

```
1 static struct sched_entity *__pick_eevdf(struct cfs_rq *cfs_rq) {
2     struct rb_node *node = cfs_rq->tasks_timeline.rb_root.rb_node;
3     struct sched_entity *best = NULL;
4     while (node) {
5         se = __node2se(node);
6         // 检查实体是否满足 eligibility 条件
7         if (entity_eligible(cfs_rq, se)) {
8             best = se;
9             node = node->rb_left; // 在左子树找更小 deadline
10        } else {
11            node = node->rb_right; // 不 eligible → 向右搜索
12        }
13    }
14    return best;
15 }
```

Listing 11: `__pick_eevdf` —EEVDF 核心选择 (kernel/sched/fair.c)

`entity_eligible()` 内部比较实体的 `vlag`：若 `vlag >= 0`，说明该实体尚未获得应得的 CPU 时间，具有 `eligibility`；若 `vlag < 0`，说明实体已超额运行，需等待其他实体追赶。

为什么 EEVDF 需要 `eligibility` 检查——CFS 的“唤醒风暴”问题。经典 CFS 按最小 `vruntime` 选进程。考虑一个场景：100 个进程每 10ms 做一次 `read()` 等待磁盘 I/O，还有一个纯 CPU 计算的进程在后台。每次 I/O 完成，唤醒的进程 `vruntime` 极小（它睡了很久，`vruntime` 没增长），于是 `pick_next_task_fair` 立刻选它、抢占当前 CPU 进程。100 个进程 × 每秒 100 次唤醒 = 每秒 100 次上下文切换，CPU 的大量时间消耗在切换本身而非实际计算。EEVDF 用 `eligibility` 解决了这个问题：刚被唤醒的进程虽然在就绪队列中，但它的 `vlag` 为负——它睡觉期间没有“消费”CPU，但从系统的角度看，它“欠”了其他一直在就绪的进程。只有等 `vlag` 归零（其他进程追赶上它的 `vruntime`），该进程才算 `eligible`，才能被 `pick_next_entity` 选中。这本质上是把调度决策从“

谁最缺 CPU” 变为” 谁既不缺 CPU、deadline 又最近”[3, 21]。

2.4.4 时钟中断与抢占检查—scheduler_tick()

每次时钟中断触发 scheduler_tick(), 调用链为 scheduler_tick() → task_tick_fair() → entity_tick():

```
1 static void entity_tick(struct cfs_rq *cfs_rq, struct sched_entity *curr) {
2     // 1. 更新当前实体的 vruntime 和 deadline 统计
3     update_curr(cfs_rq);
4     // 2. 若运行队列中不止一个任务, 检查是否需要抢占
5     if (cfs_rq->nr_running > 1)
6         check_preempt_tick(cfs_rq, curr);
7 }
```

Listing 12: entity_tick 时间片更新与抢占检查 (kernel/sched/fair.c)

update_curr() 计算当前实体的实际运行时间增量 ($\text{delta_exec} = \text{now} - \text{exec_start}$), 按负载权重缩放后累加到 vruntime, 同时更新 min_vruntime。check_preempt_tick() 比较当前实体与红黑树中最左实体的 deadline, 若当前实体的剩余时间片已耗尽或最左实体的 deadline 显著更早, 设置 TIF_NEED_RESCHED 标志。

唤醒抢占: try_to_wake_up() → check_preempt_curr() → check_preempt_wakeup()。若被唤醒进程的 deadline 早于当前进程的 deadline 且满足 eligibility 条件, 调用 resched_curr() 设置抢占标志。

2.5 上下文切换: context_switch()

context_switch() (kernel/sched/core.c) 是进程切换的中央调度函数, 完成了从 prev 进程到 next 进程的完整状态转移。该函数分两个半场:

```
1 static __always_inline struct rq *
2 context_switch(struct rq *rq, struct task_struct *prev,
3               struct task_struct *next, struct rq_flags *rf) {
4     prepare_task_switch(rq, prev, next);    // 准备阶段
5
6     // 上半场: 地址空间切换
7     arch_start_context_switch(prev);
8     switch_mm_irqs_off(prev->active_mm, next->mm, next);
9
10    // 下半场: 执行现场切换
11    switch_to(prev, next, prev);
12    // ↑ 第三个参数 prev 在返回后变为 last (从哪个进程切换来)
13    return finish_task_switch(prev);
14 }
```

Listing 13: context_switch 核心结构 (kernel/sched/core.c)

2.5.1 地址空间切换—switch_mm_irqs_off()

switch_mm_irqs_off() (arch/x86/mm/tlb.c) 完成虚拟地址空间的切换 [24]:

```
1 void switch_mm_irqs_off(struct mm_struct *prev, struct mm_struct *next,
2                          struct task_struct *tsk) {
3     if (prev == next) return;    // 内核线程, 无需切换
4     // 选择新 ASID (Address Space Identifier, PCID 特性)
```

```

5  choose_new_asid(next, next_tlb_gen, &new_asid, &need_flush);
6  // 加载新页表基址到 CR3 寄存器
7  load_new_mm_cr3(next->pgd, new_asid, need_flush);
8  this_cpu_write(cpu_tlbstate.loaded_mm, next);
9 }

```

Listing 14: switch_mm_irqs_off —CR3 切换 (arch/x86/mm/tlb.c)

load_new_mm_cr3() 通过 write_cr3() 内联汇编写入 CR3 寄存器。现代 x86 处理器支持 PCID (Process-Context Identifier)，允许 TLB 条目携带地址空间标签，避免每次 CR3 写入时全量刷新 TLB。choose_new_asid() 函数负责分配新的 ASID 并决定是否需要 TLB 刷新 [17]。

PCID 的一个常见误解需要澄清：PCID 不是“让上下文切换不刷新 TLB”——它的实际效果更微妙。没有 PCID 时，写入 CR3 刷新全部 TLB 条目，包括内核空间那些标记为 Global 的条目（它们本该对所有进程有效）。有了 PCID，每个进程拥有独立的 ASID (Address Space Identifier)，其 TLB 条目仅在 ASID 匹配时命中。这意味着：切换到新进程时不必显式刷新 TLB——旧 ASID 的条目自然不会被新 ASID 的访问匹配到；内核空间的 Global 条目在 CR3 写入后仍然存活——这些条目对性能至关重要（内核代码路径、中断处理、系统调用入口）；当切换回一个刚离开不久的进程时，其 TLB 条目可能仍在（取决于 TLB 容量），可实现部分复用。PCID 的真正价值是“让无关进程的 TLB 条目在硬件中和平共存”，而非单纯“免刷新”[17]。

2.5.2 执行现场切换—switch_to 宏 → __switch_to_asm

switch_to (arch/x86/include/asm/switch_to.h) 是执行流切换的核心宏：

```

1 #define switch_to(prev, next, last)          \
2 do {                                         \
3     ((last) = __switch_to_asm((prev), (next))); \
4 } while (0)

```

Listing 15: switch_to 宏定义 (arch/x86/include/asm/switch_to.h)

__switch_to_asm() (arch/x86/entry/entry_64.S) 是上下文切换最底层的汇编实现，执行以下五个步骤 [19]：

```

1 SYM_FUNC_START(__switch_to_asm)
2     /* %rdi = prev, %rsi = next */
3
4     /* 1. Save callee-saved registers onto prev's stack */
5     pushq   %rbp
6     pushq   %rbx
7     pushq   %r12
8     pushq   %r13
9     pushq   %r14
10    pushq   %r15
11
12    /* 2. Save prev's RSP -> prev->thread.sp */
13    movq     %rsp, TASK_threadsp(%rdi)
14
15    /* 3. Load next's RSP <- next->thread.sp */
16    movq     TASK_threadsp(%rsi), %rsp
17
18    /* EXECUTION FLOW NOW RUNS ON next's KERNEL STACK */
19
20    /* 4. Restore next's callee-saved registers */
21    popq     %r15

```

```

22     popq    %r14
23     popq    %r13
24     popq    %r12
25     popq    %rbx
26     popq    %rbp
27
28     /* 5. Jump to __switch_to() C function */
29     jmp     __switch_to
30 SYM_FUNC_END(__switch_to_asm)

```

Listing 16: `__switch_to_asm` 汇编实现 (arch/x86/entry/entry_64.S)

2.5.3 `__switch_to_asm` 寄存器状态逐帧分析

帧 0 —进入: `RDI = prev task_struct *`, `RSI = next task_struct *`, `RSP = prev` 的内核栈顶, `RIP = __switch_to_asm` 第一条指令。

帧 1-6 —保存 `prev` 现场: 依次 `pushq %rbp, %rbx, %r12, %r13, %r14, %r15`, 六次 `push` 后 `RSP` 下降 48 字节, `prev` 内核栈顶存储完整 `inactive_task_frame` 结构。该结构保证压栈/弹栈次序严格匹配, 是 `__switch_to_asm` 正确性的关键。

帧 7 —RSP 切换点: `movq %rsp, TASK_threadsp(%rdi)` 将当前 `RSP` 写入 `prev->thread.sp`。 `movq TASK_threadsp(%rsi), %rsp` 将 `RSP` 替换为 `next->thread.sp`。此时 `RSP` 指向 `next` 的内核栈——执行流已切换。

帧 8-13 —恢复 `next` 现场: 逆序弹出 `%r15, %r14, %r13, %r12, %rbx, %rbp`, 恢复 `next` 上次被切换出时的全部 callee-saved 寄存器值。

帧 14 —架构收尾: `jmp __switch_to` 进入 C 函数。`__switch_to()` (arch/x86/kernel/process_64.c) 是上下文切换的 C 语言收尾阶段, 其核心职责是将 CPU 的”管理结构”从 `prev` 切换到 `next`, 而非寄存器现场 (寄存器现场已在 `__switch_to_asm` 中完成切换):

```

1 struct task_struct *__switch_to(struct task_struct *prev,
2                                struct task_struct *next) {
3     // 1. 更新 per-CPU 的 current_task 为 next
4     this_cpu_write(current_task, next);
5
6     // 2. 切换 FPU/SIMD 状态 (XSAVE 指令保存/恢复)
7     //   — 若 prev 使用了 AVX/AMX, 保存其寄存器上下文
8     //   — 若 next 之前被切换出时 FPU 状态未脏, 则跳过恢复
9     switch_fpu(prev, next);
10
11    // 3. 切换 FS/GS 段寄存器 (线程局部存储 TLS)
12    x86_fsgsbase_load(prev, next);
13
14    // 4. 扩展状态切换: DEBUG 寄存器、I/O 权限位图
15    //     perf 事件上下文等非通用寄存器状态
16    __switch_to_xtra(prev, next);
17
18    return prev; // 返回 prev (即 switch_to 宏中的 last)
19 }

```

Listing 17: `__switch_to` C 语言收尾 (arch/x86/kernel/process_64.c)

`switch_fpu()` 是其中的性能关键点。现代处理器 (支持 XSAVE/AVX/AMX) 的 FPU 状态可达数 KB, 每次上下文切换都完整保存和恢复将带来显著开销。Linux 通过**惰性 FPU 切换优化**: `switch_fpu()` 仅将 `prev` 的 FPU 状态标记为”脏” (若其实际使用过 FPU), 而不立即保存; 只

有当下一调度选择使用 FPU 的其他进程时，才通过 `xsave` 指令保存 `prev` 的 FPU 状态到 `prev->thread.fpu`，并通过 `xrstor` 恢复 `next` 的状态。若 `prev` 从未使用过 FPU（如纯整数计算进程），整个保存步骤被省略 [18]。

上下文切换的真实成本。表面上看，`__switch_to_asm` 只做了 6 个 `push` + RSP 切换 + 6 个 `pop` + 1 个 `jmp`，约 20 条指令、几十个 `cycle`。但这只是冰山浮尖。真正的开销在切换之后：

- **TLB 失效：**CR3 写入后，若无 PCID，整个 TLB 被刷新。新进程的每次内存访问都是 TLB miss——页表遍历需要 4 次内存访问（`pgd`→`p4d`→`pud`→`pmd`→`pte`，五级页表），每次约 10–20 `cycles`。进程恢复执行后的前几十次访问几乎全部 TLB miss。
- **Cache 污染：**L1/L2/L3 cache 中填满了 `prev` 的数据。`next` 的代码和数据首次访问几乎必定 cache miss——L2 miss 约 40 `cycles`，L3 miss 约 200 `cycles`，DRAM 访问约 200ns。
- **分支预测器状态错位：**现代 CPU 的分支预测器使用全局历史（过去若干分支的 `taken/not-taken` 模式）做预测。切换到 `next` 后，分支预测器看到的是 `prev` 的历史、执行的是 `next` 的代码——前几十个分支几乎随机预测，平均 2–3 次误预测（每次约 20 `cycles` 的流水线冲刷）。
- **综合估算：**一次完整的上下文切换（含 CR3 写入），在无 PCID 的 x86_64 处理器上，真实开销在 3,000–10,000 `cycles`。其中 `__switch_to_asm` 的指令执行本身占比不到 1%。引入 PCID 后 TLB 刷新被避免，但 cache 污染和分支预测器偏移仍然存在 [17]。

上下文切换的完整调用时序如图 2 所示。

2.6 完整调度调用链

从 `schedule()` 到 `__switch_to()` 的完整调用链如图 3 所示。

2.7 结构体关系

图 4 展示了 `task_struct`、`sched_entity`、`cfs_rq`、`rq`、`thread_struct` 之间的包含与引用关系。图 5 展示了调度类层次结构的遍历顺序。

2.8 调度类实战：SCHED_MFQ 完整实现分析

前文从通用架构层面分析了 Linux 调度器的层次结构和调用链。本节以实验 11 中实际实现的 SCHED_MFQ（Multi-level Feedback Queue，多级反馈队列，`policy=4`）调度类为完整案例，展示如何从零开始实现一个新的调度类并将其集成到 Linux 6.18.15 内核中。该案例覆盖了调度类注册、数据结构嵌入、全部回调函数实现、内核编译集成以及用户态调用验证的完整链路，是对前文所有调度理论分析的工程验证。

2.8.1 MFQ 算法设计

MFQ 由 Fernando J. Corbató 于 1962 年在 CTSS 系统中首次提出 [28]。本实现采用 8 级队列（L0–L7），每级时间片为 `10 << level ticks`：

核心规则：

1. 新进程进入 L0（最高优先级）。
2. 时间片耗尽且队列中有其他进程 → 降级（`level++`），时间片翻倍。

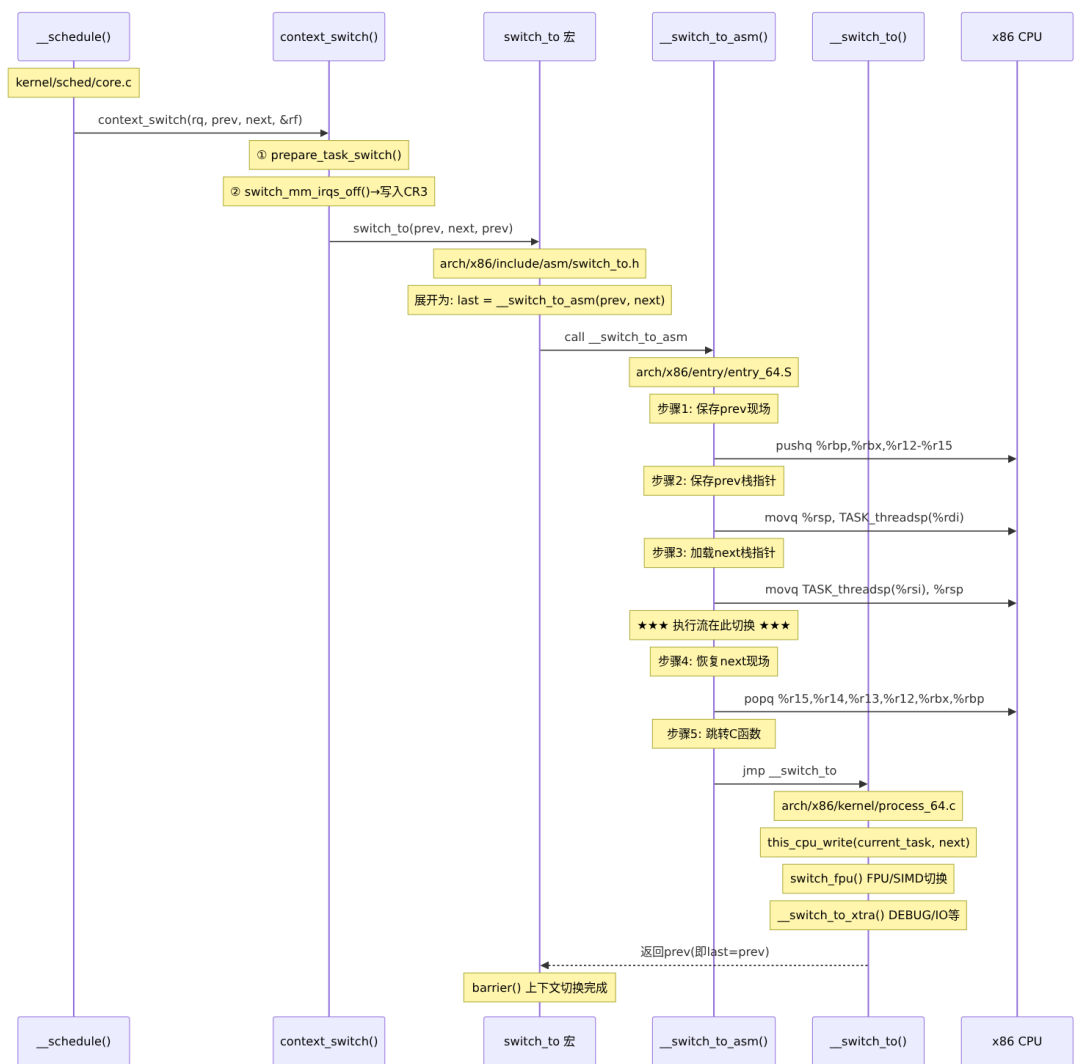


图 2: __switch_to_asm 寄存器级切换时序

表 4: SCHED_MFQ 队列参数

队列	时间片 (ticks)	目标进程类型
L0 (最高优先级)	10	新进程、I/O 密集型
L1	20	轻度 CPU 密集型
L2	40	—
L3	80	—
L4	160	—
L5	320	—
L6	640	—
L7 (最低优先级)	1280	纯 CPU 密集型后台任务

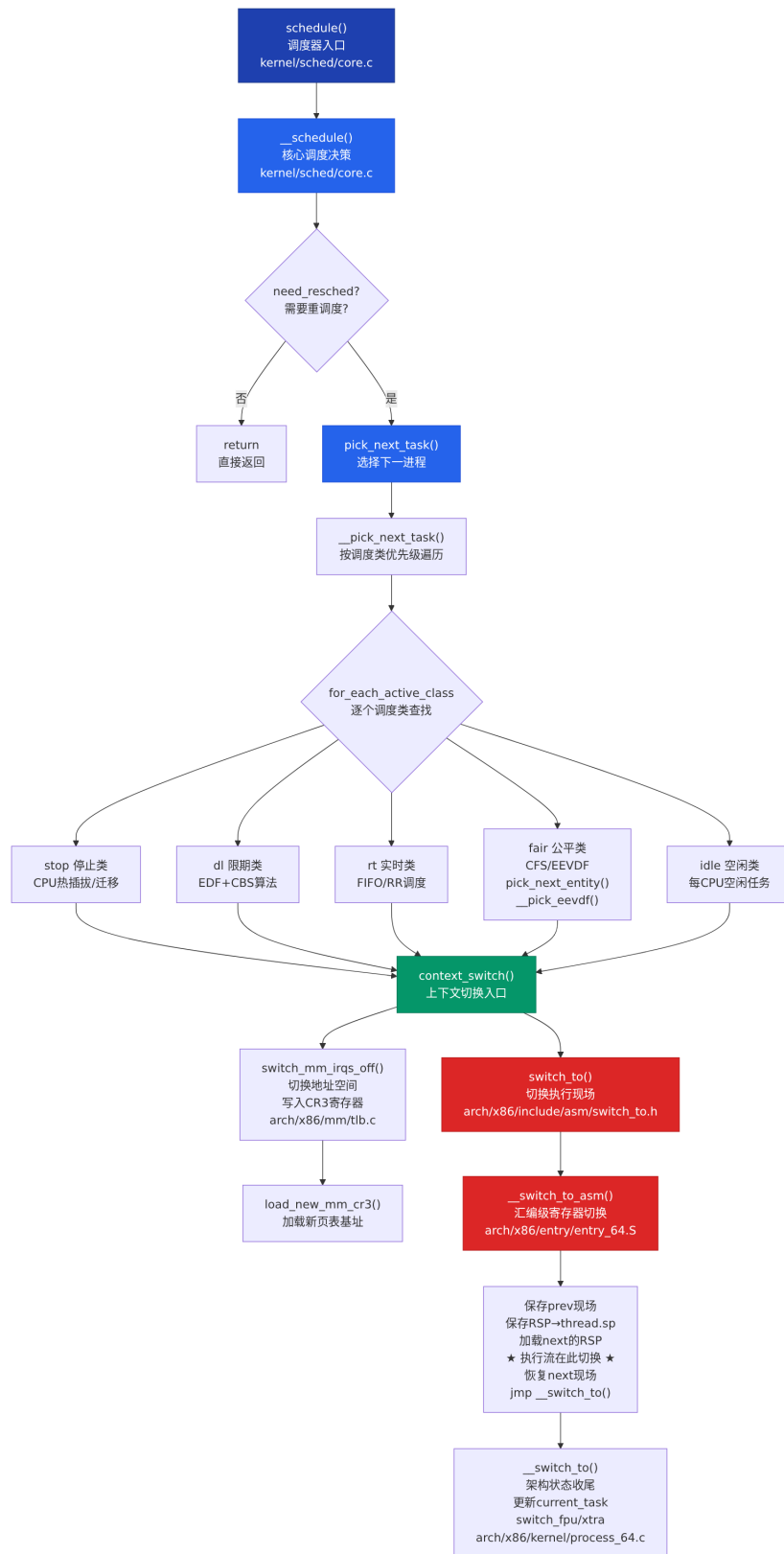


图 3: `schedule()` → `__schedule()` → `context_switch()` 完整调用链

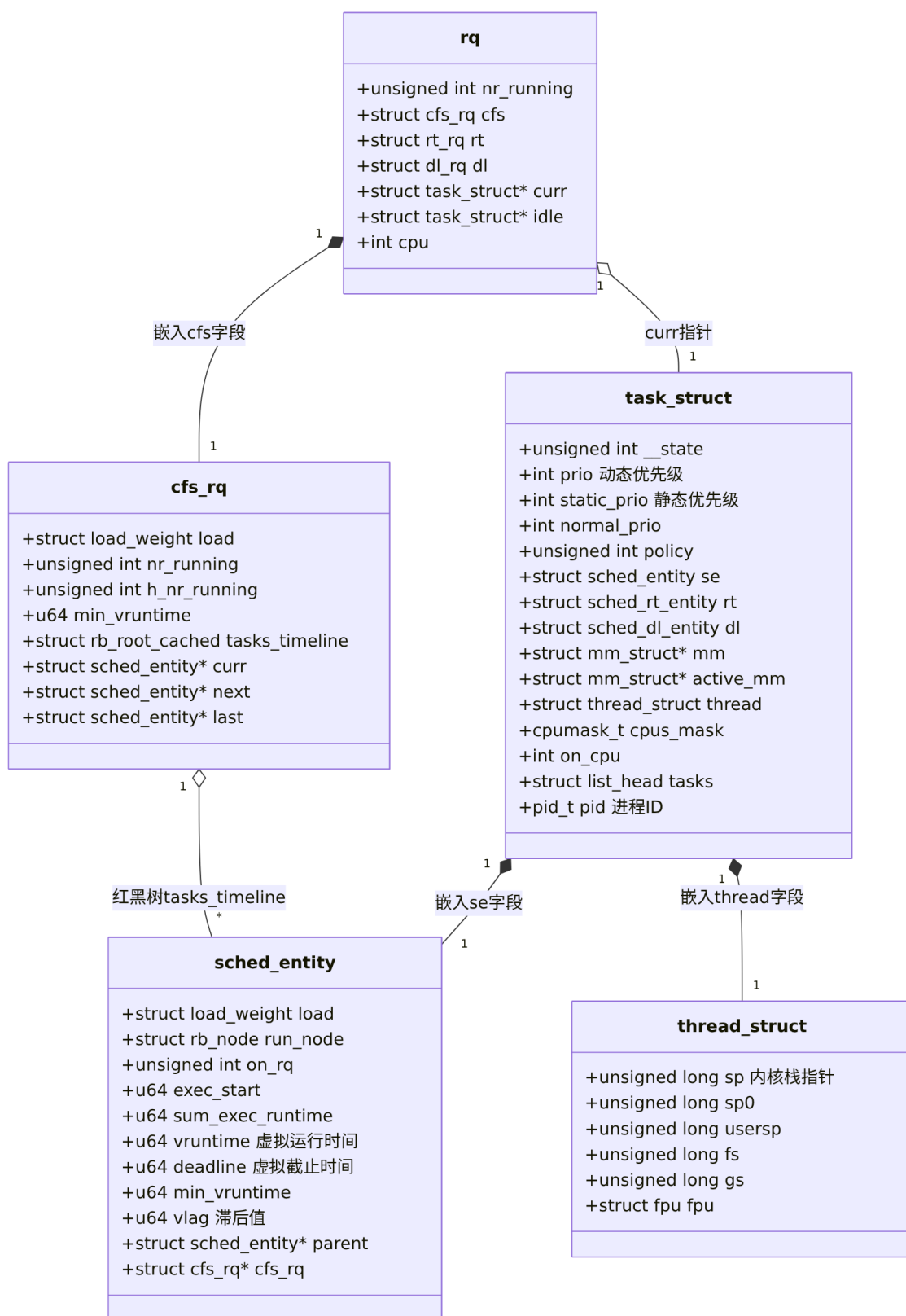


图 4: task_struct / sched_entity / cfs_rq / rq 结构体关系

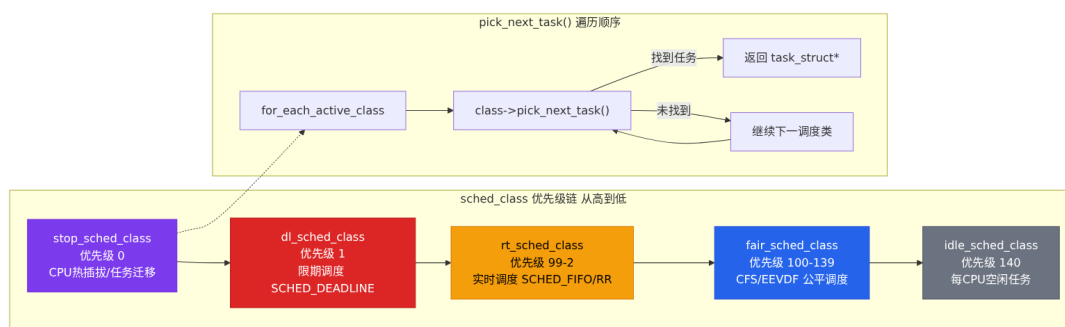


图 5: 调度类优先级层次与 pick_next_task 遍历顺序

3. 队列中唯一进程 → 不降级（降级无意义）。
4. I/O 同步唤醒（WF_SYNC 标志）→ 升级（level+），并可能抢占当前进程。

2.8.2 内核数据结构嵌入

实现新调度类的第一步是在内核核心数据结构中嵌入调度器私有字段。需要修改三个文件：

include/linux/sched.h —— 在 **task_struct** 中嵌入 **sched_mfq_entity**：

```
1 // include/linux/sched.h, line ~880
2 struct task_struct {
3     // ... 其他字段 ...
4     struct sched_entity se;           // CFS 调度实体
5 #ifdef CONFIG_SCHED_MFQ
6     struct sched_mfq_entity mfq_se;   // MFQ 调度实体(新增)
7 #endif
8     struct sched_rt_entity rt;        // RT 调度实体
9     // ...
10 };
11
12 // 同时需要定义 sched_mfq_entity 结构体:
13 struct sched_mfq_entity {
14     struct list_head run_list;        // 链接到 mfq_rq.queues[level]
15     int queue_level;                 // 当前队列级别 (0-7)
16     u64 time_slice;                  // 剩余时间片 (ticks)
17 };
```

Listing 18: task_struct 中的 MFQ 实体嵌入 (include/linux/sched.h)

kernel/sched/sched.h —— 定义 per-CPU 的 MFQ 运行队列和调度类声明：

```
1 // kernel/sched/sched.h
2 #define MFQ_QUEUE_LEVELS 8
3
4 struct mfq_rq {
5     raw_spinlock_t lock;              // 保护本 MFQ 队列的锁
6     unsigned int nr_running;          // 当前在队列中的任务数
7     struct list_head queues[MFQ_QUEUE_LEVELS]; // 8级就绪队列
8     struct task_struct *curr;         // 当前在此CPU运行的MFQ任务
9 };
10
11 DECLARE_PER_CPU(struct mfq_rq, mfq_rq); // per-CPU 变量声明
12 extern const struct sched_class mfq_sched_class; // 调度类外部声明
```

Listing 19: mfq_rq 结构和调度类声明 (kernel/sched/sched.h)

`include/uapi/linux/sched.h` ——注册调度策略常量:

```
1 #define SCHED_MFQ      4      // MFQ 调度策略号 (新增)
```

Listing 20: SCHED_MFQ 策略注册 (`include/uapi/linux/sched.h`)

2.8.3 调度类注册与集成

新调度类需要在内核的三个关键位置注册, 使其参与调度决策:

调度类路由 (`kernel/sched/core.c`): 在 `__setscheduler_class()` 中添加 MFQ 分支, 使 `sched_setscheduler()` 系统调用能将任务的调度策略设置为 SCHED_MFQ:

```
1 static const struct sched_class *__setscheduler_class(int policy) {
2     // ...stop/dl/rt 分支...
3 #ifdef CONFIG_SCHED_MFQ
4     if (policy == 4) /* SCHED_MFQ */
5         return &mfq_sched_class;
6 #endif
7     return &fair_sched_class; // 默认走 CFS
8 }
```

Listing 21: 调度类路由——`__setscheduler_class` (`kernel/sched/core.c`)

`valid_policy()` 更新 (`kernel/sched/sched.h`): 使 `sched_setscheduler()` 的合法性检查接受 `policy=4`:

```
1 #ifdef CONFIG_SCHED_MFQ
2 static inline int mfq_policy(int policy)
3 {
4     return policy == SCHED_MFQ;
5 }
6 #else
7 static inline int mfq_policy(int policy) { return 0; }
8 #endif
9 // valid_policy() 内部增加: || mfq_policy(policy)
```

Listing 22: `valid_policy` 与 `mfq_policy` 辅助函数 (`kernel/sched/sched.h`)

链接脚本顺序 (`include/asm-generic/vmlinux.lds.h`): 将 `mfq_sched_class` 放置在 RT 和 CFS 之间, 保证调度类遍历顺序为 `stop→dl→rt→mfq→fair→idle`:

```
1 *(__rt_sched_class)
2 *(__mfq_sched_class) // 插入在 rt 和 fair 之间
3 *(__fair_sched_class)
```

Listing 23: 链接脚本中的 MFQ 位置 (`include/asm-generic/vmlinux.lds.h`)

编译集成 (`kernel/sched/Makefile` + `init/Kconfig`): 添加 `CONFIG_SCHED_MFQ` 配置选项和编译规则。

2.8.4 调度器初始化

MFQ 调度器在系统启动时通过 `sched_init()` 初始化。每个 CPU 的 `mfq_rq` 通过 `DEFINE_PER_CPU` 自动分配并清零:

```
1 // kernel/sched/core.c, sched_init() 函数中
2 #ifdef CONFIG_SCHED_MFQ
```

```

3 for_each_possible_cpu(i) {
4     struct mfq_rq *mrq = &per_cpu(mfq_rq, i);
5     raw_spin_lock_init(&mrq->lock);
6     mrq->nr_running = 0;
7     mrq->curr = NULL;
8     for (int l = 0; l < MFQ_QUEUE_LEVELS; l++)
9         INIT_LIST_HEAD(&mrq->queues[l]);
10 }
11 #endif

```

Listing 24: MFQ 调度器初始化 (kernel/sched/core.c)

2.8.5 完整回调函数逐行分析

以下按照调度器调用链的实际发生顺序，逐函数分析 sched_mfq.c (完整 135 行) 的实现逻辑。

enqueue_task_mfq ——任务入队 当 wake_up_new_task() 或 try_to_wake_up() 将任务加入调度队列时调用。MFQ 入队逻辑的核心是在任务首次被调度时自动初始化其 MFQ 实体 (防御性设计)，然后按当前 queue_level 将任务加入对应优先级队列的尾部 (FIFO 语义)：

```

1 static void enqueue_task_mfq(struct rq *rq, struct task_struct *p, int flags)
2 {
3     struct mfq_rq *mrq = &per_cpu(mfq_rq, cpu_of(rq));
4     struct sched_mfq_entity *mse = &p->mfq_se;
5     // 防御性初始化：若 run_list 从未初始化 (如被迁移的任务)，
6     // 自动调用 mfq_entity_init()。该检查保证不会因遗漏初始化而
7     // 导致 list_add_tail 操作未初始化的链表指针 → 内核崩溃
8     if (unlikely(mse->run_list.next == NULL))
9         mfq_entity_init(mse);
10    raw_spin_lock(&mrq->lock);
11    // 将任务加入其当前 queue_level 对应队列的尾部
12    // FIFO 入队：同一优先级队列内按到达顺序调度
13    list_add_tail(&mse->run_list, &mrq->queues[mse->queue_level]);
14    mrq->nr_running++;
15    raw_spin_unlock(&mrq->lock);
16 }

```

Listing 25: enqueue_task_mfq (kernel/sched/sched_mfq.c)

设计要点：mfq_entity_init() 将 queue_level 设为 0、time_slice 设为 10 ticks——这意味着新进程总是从最高优先级 L0 开始，获得最短的时间片。这是 MFQ 算法的核心机制：初始假设所有进程都是交互型的 (短时间片快速响应)，仅当进程用完时间片而不阻塞时才逐渐降级。

dequeue_task_mfq ——任务出队 当任务因阻塞、退出或被迁移到其他 CPU 而需要从调度队列中移除时调用：

```

1 static bool dequeue_task_mfq(struct rq *rq, struct task_struct *p, int flags)
2 {
3     struct mfq_rq *mrq = &per_cpu(mfq_rq, cpu_of(rq));
4     raw_spin_lock(&mrq->lock);
5     // 安全移除：先检查 run_list 非空再执行 list_del_init
6     if (!list_empty(&p->mfq_se.run_list)) {
7         list_del_init(&p->mfq_se.run_list); // 删除并重新初始化链表节点
8         mrq->nr_running--;
9     }
10    raw_spin_unlock(&mrq->lock);

```



```

11     return true;
12 }

```

Listing 26: dequeue_task_mfq (kernel/sched/sched_mfq.c)

pick_task_mfq —— **核心选择逻辑** 这是 MFQ 调度器最核心的函数。调度器从 L0 到 L7 扫描，返回第一个非空队列的队首任务。时间复杂度 $O(8) = O(1)$ ，与队列中的任务数无关：

```

1 static struct task_struct *pick_task_mfq(struct rq *rq)
2 {
3     struct mfq_rq *mrq = &per_cpu(mfq_rq, cpu_of(rq));
4     struct task_struct *p = NULL;
5     int level;
6     raw_spin_lock(&mrq->lock);
7     // 核心：从 L0 到 L7 扫描，严格优先级
8     for (level = 0; level < 8; level++) {
9         if (!list_empty(&mrq->queues[level])) {
10             // FIFO 出队：取队首（最早入队的任务）
11             p = list_first_entry(&mrq->queues[level],
12                                 struct task_struct, mfq_se.run_list);
13             goto out;
14         }
15     }
16 out:
17     mrq->curr = p; // 记录当前选中的任务
18     raw_spin_unlock(&mrq->lock);
19     return p;
20 }

```

Listing 27: pick_task_mfq (kernel/sched/sched_mfq.c)

关键设计决策——为何使用 pick_task 而非 pick_next_task? Linux 核心调度器对实现了 pick_next_task 的调度类有特殊约定：该类必须自行完成 put_prev_task 和 set_next_task 的全部操作。RT 调度器仅实现了 pick_task，将 put_prev/set_next 交由核心的 put_prev_set_next_task() 统一处理。MFQ 遵循同样模式，避免了在 pick_next_task 中重复实现队列维护逻辑，降低了 bug 风险。

put_prev_task_mfq 与 set_next_task_mfq ——任务状态切换 put_prev_task_mfq 在 prev 进程被换出 CPU 时调用。本实现中该函数仅做 lock/unlock 以维护 preempt_count 的 spinlock 跟踪平衡，不操作队列：

```

1 static void put_prev_task_mfq(struct rq *rq, struct task_struct *prev,
2                               struct task_struct *next)
3 {
4     struct mfq_rq *mrq = &per_cpu(mfq_rq, cpu_of(rq));
5     if (prev == next) return; // 无需切换
6     raw_spin_lock(&mrq->lock); // 维护 preempt_count
7     raw_spin_unlock(&mrq->lock);
8 }

```

Listing 28: put_prev_task_mfq (kernel/sched/sched_mfq.c)

set_next_task_mfq 在 next 被正式设为当前运行任务时调用，负责将其从就绪队列中移除（因为它此刻正在运行）：

```

1 static void set_next_task_mfq(struct rq *rq, struct task_struct *p, bool first)
2 {
3     struct mfq_rq *mrq = &per_cpu(mfq_rq, cpu_of(rq));
4     raw_spin_lock(&mrq->lock);
5     // 从就绪队列中移除——它正在 CPU 上运行，不应再在队列中
6     if (!list_empty(&p->mfq_se.run_list)) {
7         list_del_init(&p->mfq_se.run_list);
8         mrq->nr_running--;
9     }
10    mrq->curr = p; // 标记为当前运行任务
11    raw_spin_unlock(&mrq->lock);
12 }

```

Listing 29: set_next_task_mfq (kernel/sched/sched_mfq.c)

task_tick_mfq ——时钟中断与时间片管理 每次时钟中断触发 scheduler_tick() → task_tick_mfq(), 完成时间片递减和降级判断:

```

1 static void task_tick_mfq(struct rq *rq, struct task_struct *curr, int queued)
2 {
3     struct mfq_rq *mrq = &per_cpu(mfq_rq, cpu_of(rq));
4     struct sched_mfq_entity *mse = &curr->mfq_se;
5     raw_spin_lock(&mrq->lock);
6     // 消耗 1 tick
7     if (mse->time_slice > 0)
8         mse->time_slice--;
9     // 时间片耗尽 → 降级逻辑
10    if (mse->time_slice == 0) {
11        // 条件1: 有其他进程在等待 (否则降级无意义)
12        // 条件2: 尚未到达最低队列 L7 (无更低队列可降)
13        if (mrq->nr_running > 0 && mse->queue_level < 7) {
14            mse->queue_level++; // 降级
15            resched_curr(rq); // 设置 TIF_NEED_RESCHED 触发调度
16        }
17        // 重新分配下一级对应的的时间片
18        mse->time_slice = mfq_get_timeslice(mse->queue_level);
19    }
20    raw_spin_unlock(&mrq->lock);
21 }

```

Listing 30: task_tick_mfq (kernel/sched/sched_mfq.c)

resched_curr(rq) 设置 TIF_NEED_RESCHED 标志。CPU 在下一个抢占检查点 (如系统调用返回或中断返回) 检测到此标志后, 核心调度器调用 schedule() → __schedule() → pick_next_task() → pick_task_mfq(), 再次扫描队列选择新任务。

wakeup_preempt_mfq ——唤醒升级与抢占 当 I/O 完成或其他事件唤醒一个 MFQ 任务时, check_preempt_curr() → wakeup_preempt_mfq() 被调用。这是 MFQ”反馈”机制的核心——I/O 密集型任务获得优先级提升:

```

1 static void wakeup_preempt_mfq(struct rq *rq, struct task_struct *p,
2                               int flags)
3 {
4     // 仅当当前运行任务也是 MFQ 任务时才进行抢占判断
5     if (rq->donor->sched_class != &mfq_sched_class)
6         return;

```

```

7   struct sched_mfq_entity *mse = &p->mfq_se;
8   struct task_struct *curr = rq->donor;
9   // I/O 同步唤醒 → 升级 (提升队列级别)
10  if (mse->queue_level > 0 && (flags & WF_SYNC)) {
11      mse->queue_level--; // 向上升一级
12      mse->time_slice = mfq_get_timeslice(mse->queue_level);
13  }
14  // 若被唤醒进程的优先级高于当前进程 → 抢占
15  if (mse->queue_level < curr->mfq_se.queue_level)
16      resched_curr(rq);
17 }

```

Listing 31: wakeup_preempt_mfq (kernel/sched/sched_mfq.c)

WF_SYNC 标志表示这是一个“同步唤醒”——即唤醒者（如中断处理程序）直接进入了被唤醒进程的等待路径，被唤醒进程极可能立即执行（而非仅加入就绪队列等待）。典型的同步唤醒场景包括：管道写入者唤醒等待在管道上的读取者；磁盘 I/O 完成中断唤醒等待该 I/O 的进程。

task_fork_mfq / switched_to_mfq ——生命周期管理 task_fork_mfq 在 fork() 创建新进程时调用，初始化 MFQ 实体 (queue_level=0, time_slice=10)：

```

1 static void task_fork_mfq(struct task_struct *p) {
2     mfq_entity_init(&p->mfq_se); // L0, 10 ticks
3 }
4
5 // switched_to: 进程通过 sched_setscheduler 切换到 SCHED_MFQ 时调用
6 static void switched_to_mfq(struct rq *rq, struct task_struct *p) {
7     mfq_entity_init(&p->mfq_se); // 重新初始化, 从 L0 开始
8 }
9
10 // switched_from: 进程从 SCHED_MFQ 切换到其他调度类时调用
11 static void switched_from_mfq(struct rq *rq, struct task_struct *p) {
12     struct mfq_rq *mrq = &per_cpu(mfq_rq, cpu_of(rq));
13     raw_spin_lock(&mrq->lock);
14     if (!list_empty(&p->mfq_se.run_list)) {
15         list_del_init(&p->mfq_se.run_list); // 从 MFQ 队列中移除
16         mrq->nr_running--;
17     }
18     raw_spin_unlock(&mrq->lock);
19 }

```

Listing 32: task_fork / switched_to / switched_from (kernel/sched/sched_mfq.c)

DEFINE_SCHED_CLASS(mfq) ——调度类实例化 所有回调函数通过 DEFINE_SCHED_CLASS 宏组装为 mfq_sched_class 实例。宏定义位于 kernel/sched/sched.h，自动生成 const struct sched_class mfq_sched_class 并将所有回调指针填入对应槽位：

```

1 DEFINE_SCHED_CLASS(mfq) = {
2     .enqueue_task      = enqueue_task_mfq,
3     .dequeue_task      = dequeue_task_mfq,
4     .wakeup_preempt    = wakeup_preempt_mfq,
5     .pick_task         = pick_task_mfq,
6     .put_prev_task     = put_prev_task_mfq,
7     .set_next_task     = set_next_task_mfq,
8     .select_task_rq    = select_task_rq_mfq,
9     .migrate_task_rq   = migrate_task_rq_mfq,

```

```

10 .task_tick      = task_tick_mfq,
11 .task_fork     = task_fork_mfq,
12 .task_dead     = task_dead_mfq,
13 .switched_from = switched_from_mfq,
14 .switched_to   = switched_to_mfq,
15 .update_curr   = update_curr_mfq,      // no-op
16 .set_cpus_allowed = set_cpus_allowed_mfq, // no-op
17 .prio_changed  = prio_changed_mfq,    // no-op
18 };

```

Listing 33: DEFINE_SCHED_CLASS(mfq) 实例化 (kernel/sched/sched_mfq.c)

其中 `update_curr`, `set_cpus_allowed`, `prio_changed` 均为空操作——核心调度器无条件调用这些回调，但 MFQ 不需要这些钩子执行实际逻辑，因此提供空函数以满足接口要求。`select_task_rq_mfq` 实现了简单的 CPU 选择：若为唤醒路径 (WF_TTWU) 且任务有记录的 last-run CPU (`wake_cpu`)，则选择该 CPU (利用 cache 热度)；否则保持在当前 CPU。

2.8.6 完整调用链示例：一次抢占切换的逐层追踪

以下选择具体场景来展示 MFQ 调度器在一次抢占切换中的完整调用链：P1 是 CPU-bound 进程，已降级至 L3 (时间片 80 ticks, 剩余 1 tick)；P2 是 I/O-bound 进程，刚完成 I/O 被唤醒至 L0 (时间片 10 ticks)。当时钟中断触发 P1 时间片耗尽时，从硬件到调度器到寄存器切换的完整时序如下：

这张表把五层 (硬件 → 中断 → 核心调度器 → MFQ 调度器 → 用户进程) 放在一根时间轴上，每层的操作在同一行中对齐。其中第 8 行的 RSP 切换是整个链路最关键的一个指令对——在此之前，CPU 运行 P1 的内核代码；在此之后，CPU 运行 P2 的内核代码。P1 在 `mov %rsp, prev->thread.sp` 之后被“冻住”，它的内核栈上压着 `r15/r14/r13/r12/rbx/rbp` 六个寄存器的值，`prev->thread.sp` 指向那个位置。将来某次调度器再次选中 P1 时，`__switch_to_asm` 的后半段会从 `next->thread.sp` (那时 P1 是 next) 恢复 RSP，依次弹出六个寄存器，`jmp __switch_to` → 一路返回到 `context_switch` → `__schedule` → `schedule`，P1 的时间继续流动——它对中间发生的一切毫无记忆。

2.8.7 用户态调用

由于 glibc 的 `sched_setscheduler()` 包装函数拒绝未知的 policy 值 (仅允许 `SCHED_FIFO`/`SCHED_RR`/`SCHED_NORMAL` 等标准策略)，用户态程序必须通过 `syscall()` 直接调用内核：

```

1 #define SCHED_MFQ 4
2 struct sched_param param = { .sched_priority = 0 };
3 // 绕过 glibc, 直接使用 raw syscall
4 syscall(__NR_sched_setscheduler, 0, SCHED_MFQ, &param);
5 // 此后通过 fork 创建的子进程自动继承 SCHED_MFQ 策略
6 for (int i = 0; i < 4; i++) {
7     if (fork() == 0) {
8         // 子进程: 2 个 CPU-bound (纯计算) + 2 个 I/O-bound (定时 sleep)
9         if (i < 2) { /* CPU-bound: while(1) 空循环 */ }
10        else      { /* I/O-bound: 循环中 usleep(1000) */ }
11    }
12 }
13 // 父进程 waitpid 等待所有子进程退出

```

Listing 34: MFQ 用户态测试程序核心片段

表 5: MFQ 调度抢占切换全时序—P1(L3,CPU-bound) → P2(L0,I/O-bound)

#	硬件层	中断/抢占	核心调度器	MFQ 调度器	用户进程
1	APIC 定时器中断, CPU 保存 RIP/RFLAGS 到内核栈	<code>scheduler_tick()</code> 入口, <code>rq->lock</code> 已持	调用 <code>curr->sched_class->task_tick</code>	<code>task_tick_mfq(P1):</code> <code>time_slice--</code> → 0 → <code>queue_level=3+4</code> → <code>resched_curr()</code> 设 <code>TIF_NEED_RESCHED</code>	P1 正在用户态执行计算循环
2	—	中断返回路径: 调用 <code>exit_to_user_mode_loop()</code> 检测 <code>TIF_NEED_RESCHED</code>	调用 <code>schedule()</code> , <code>prev=P1</code>	—	—
3	—	—	<code>pick_next_task()</code> : 快速路径不命中 (<code>prev</code> 非 CFS), 进入 <code>__pick_next_task()</code>	遍历 <code>stop→dl→rt</code> 均为空, <code>for_each_active_class</code> 走到 <code>mfq</code>	—
4	—	—	—	<code>pick_task_mfq()</code> : 扫描 L0→L7, L0 非空 (P2 被唤醒在这里), <code>list_first_entry</code> 取 P2	—
5	—	—	<code>context_switch(P1, P2)</code> 入口	<code>put_prev_task_mfq</code> : lock/unlock 维护计数	P1 即将被”冻结”
6	—	—	—	<code>set_next_task_mfq(P2)</code> : <code>list_del_init</code> 将 P2 从 L0 队列移除 (标记为正在运行)	—
7	<code>write_cr3(next->pgd)</code> : 切换地址空间, 若 P2 的 mm 不同于 P1, TLB 开始渐进刷	<code>switch_mm_irqs_off()</code> → <code>load_new_mm_cr3()</code>	—	—	—
8	<code>mov %rsp, prev->thread.sp</code> : 保存 P1 的 RSP; <code>mov next->thread.sp, %rsp</code> : 切换到 P2 的内核栈 ← RSP 切换点	<code>switch_to</code> → <code>__switch_to_asm</code> : <code>push r15..rbp</code> (P1 的), 切换到 P2 栈, <code>pop r15..rbp</code> (P2 的)	—	—	P1 在此冻结, P2 开始恢复
9	<code>jmp __switch_to</code> → 更新 <code>current_task</code> 为 P2 → 切换 FPU/FSGSBASE	<code>context_switch</code> 返回 → <code>__schedule</code> 返回 → <code>schedule</code> 返回	—	—	P2 从中断返回, 回到用户态继续执行 I/O 循环 (它之前调用 <code>usleep</code> 阻塞)

验证结果：I/O-bound 子进程由于频繁 sleep→wakeup 触发 wakeup_preempt_mfq 升级，始终保持 L0/L1 高优先级，优先获得 CPU；CPU-bound 子进程逐级降级至 L4-L7，在后台完成批量计算。所有子进程正常退出，验证了从调度类注册到用户态调用的完整链路。

2.8.8 调度器接口契约：从四个典型错误理解内核设计意图

调度器回调接口的语义并不总是能从函数名直接推断。以下四个在实现 MFQ 过程中暴露的问题，每个都对应一个非显式的内核设计约束，理解这些约束对阅读任何调度器源码都有帮助：

表 6: MFQ 实验做的时候遇到的一些问题和思考

Bug	表面症状	根因
#1 实现 pick_next_task → 全系统死锁	任务被选中后既不从队列移除也不设 curr，队列状态逐渐损坏	核心调度器的契约式接口：有 pick_next_task 的调度类必须自我管理队列（put_prev + set_next 全包）。RT 调度器只实现 pick_task（不含 next），把队列维护委托给核心的 put_prev_set_next_task()。MFQ 跟随 RT 模式，删掉 pick_next_task 改用 pick_task，死锁解除。接口的存在与否本身就是信号。
#2 task_tick_mfq 内调 pr_info → scheduling while atomic panic	内核 BUG 在 core.c:6956, preempt_count 异常	原子上下文铁律：定时器中断已持有 rq->lock，MFQ 的 mrq->lock 嵌套在内——双 spinlock 上下文中 printk 可能触发 console 驱动 → 尝试获取某锁 → 若锁被其他进程持有则调度 → panic。在内核热路径中，所有可能触发调度的操作（内存分配、日志输出、copy_to/from_user）都是禁区。删掉全部 pr_info 后通过。
#3 put_prev_task_mfq 的 if(RUNNING) {list ops} → preempt_count 泄露 1	进程退出时 do_task_dead → BUG(), exited with preempt_count 1	PREEMPT_DYNAMIC 的复杂性：即使在 if(prev->__state == TASK_RUNNING) 条件体中放入 list 操作（且 body 从不执行——prev 是 TASK_DEAD），编译器生成的地址计算代码仍可能干扰 preempt_count 的锁追踪。通过 QEMU 对照实验精确缩小到“lock + if(RUNNING){list_del_init; list_add_tail} + unlock”的组合 → 崩溃；替换 body 为 barrier() → 通过。当前方案只做 lock/unlock。这是真实内核调试中最诡异的一类 bug。
#4 sched_setscheduler(SCHED_MFQ) → EINVAL	glibc 拒绝 policy=4	内核-用户态的纵深防御：valid_policy() 是内核侧防御线——它决定哪些 policy 值被接受。glibc 的 sched_setscheduler() 包装函数另有自己的白名单——它拒绝未知值，根本不到内核。这意味着自定义调度策略需要经过三道栅栏：UAPI 头定义 (#define SCHED_MFQ 4)；内核 valid_policy() 扩展；用户态用 syscall() 绕过 glibc。每道栅栏独立运行——纵深防御。

这四个问题从不同角度揭示了内核调度子系统的设计约束：接口契约决定调度类的职责边界 (Bug 1)，原子上下文限制热路径中的可调用函数集 (Bug 2)，PREEMPT_DYNAMIC 与 DEBUG_PREEMPT 的交互影响锁追踪行为 (Bug 3)，内核-用户态之间的多层校验构成纵深防御 (Bug 4)。对调度器源码的任何修改都不能脱离这些约束来理解。

2.9 实验验证

实验 11 (MFQ 调度器实现)：实现了 SCHED_MFQ (policy=4) 调度类，8 级多级反馈队列 (L0-L7，时间片 10 << level ticks)，修改涉及 8 个内核文件。QEMU 验证通过：I/O-bound 进程保持高优先级先完成，CPU-bound 进程逐级降级后完成。关键结论：自定义调度类的核心不在于代码量（仅 135 行），而在于正确理解调度类接口契约——pick_task 与 pick_next_task 的语义差异决定了队列维护的职责归属，put_prev/set_next 的锁操作必须严格匹配以维护 preempt_count 平衡。

3 进程间通信机制

3.1 System V IPC 概述

Linux 支持三种 System V IPC 机制 [15]，均通过 `ipc/` 子目录下的源码实现：**信号量** (`ipc/sem.c`，同步互斥)、**消息队列** (`ipc/msg.c`，类型化消息传递)、**共享内存** (`ipc/shm.c`，零拷贝最快 IPC)。三种 IPC 资源的生命周期管理统一由 `ipc/util.c` 中的通用 IPC ID 分配机制管理。

图 6 展示了 Linux 内核支持的 IPC 机制全景。

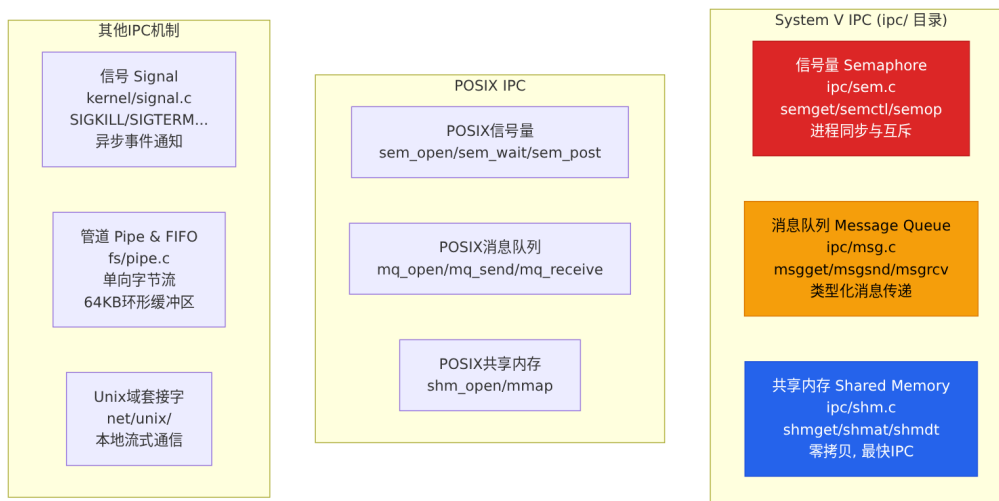


图 6: Linux 内核 IPC 机制总览

3.2 信号量 (Semaphore)

3.2.1 数据结构

```
1 // ipc/sem.c
2 struct sem_array {
3     struct kern_ipc_perm sem_perm;    // IPC 权限结构
4     time64_t sem_ctime;               // 最后修改时间
5     struct sem sems[];                // 柔性数组: 信号量数组
6     struct list_head pending_alter;    // 等待 V 操作的队列链表
7     struct list_head pending_const;    // 等待 O 操作的队列链表
8     int complex_count;
9 };
10 struct sem {
11     int semval;                       // 当前信号量值
12     int sempid;                       // 最后操作该信号量的进程 PID
13 };
```

Listing 35: `sem_array` 信号量集结构 (`ipc/sem.c`)

3.2.2 `semop()` —P/V 操作调用链

`semop()` 系统调用 (`ipc/sem.c`) 是信号量操作的核心入口。用户态通过 `semop(semid, sops, nsops)` 传递一个操作数组，内核需要保证这个操作数组的**原子性**——要么全部成功，要么全部不执

行。这一语义是信号量实现进程间互斥同步的基础，也是 Dijkstra 经典 P/V 操作理论在内核中的直接映射。

系统调用入口 `sys_semop()` → `do_semop()` 首先通过 `sem_obtain_object_check()` 根据 `semid` 查找 `struct sem_array`，然后分配 `struct sem_queue` 描述本次操作请求并将其挂入 `sem_array` 的等待队列。核心决策函数 `perform_atomic_semop()` 遍历 `semop` 数组，对每个操作：若 `sop > 0` (V 操作) 无条件将 `semval += sop`；若 `sop < 0` (P 操作) 检查 `semval >= |sop|`，满足则 `semval -= sop`，否则标记失败。若有任一 P 操作失败，回滚所有已执行的 V 操作，恢复信号量原值。

若原子操作全部成功，立即返回用户态。若失败，调用进程被设置为 `TASK_INTERRUPTIBLE` 状态，通过 `schedule_timeout()` 让出 CPU 进入睡眠，等待后续 V 操作唤醒。当其他进程执行 `semop()` 中的 V 操作后，`update_sem_otime()` 遍历 `pending_alter` 队列，对每个等待进程重新尝试 `perform_atomic_semop()`，成功则调用 `__wake_up_common()` 将其唤醒。这一”完成后重试所有等待者”的机制保证了信号量的公平性——不会因为队列顺序导致某些进程饿死。

3.2.3 从 Dijkstra PV 到 Linux semop：三层映射

课本上的 PV 原语是抽象符号，实验中的 `sem_p/sem_v` 是用户态包装函数，内核中的 `perform_atomic_semop` 是真正执行者。以下从理论到实现做三层对照，理清每一层的语义变化：

表 7: Dijkstra PV 原语 → XSI 信号量集 → 内核实现三层映射		
层次	P 操作 (减量)	V 操作 (增量)
理论层 (Dijkstra, 1965)	P(S): while(S<=0) wait; S-- ——若资源不足则自旋等待，递减必须是原子操作	V(S): S++——释放资源，若有等待者则唤醒
用户态 (exp4 的 <code>sem_p/sem_v</code>)	<code>sb.sem_op--1; semop(semid, &sb, 1)</code> ——构造 <code>sembuf</code> 结构体， <code>sem_op</code> 为负表示 P	<code>sb.sem_op++1; semop(semid, &sb, 1)</code> —— <code>sem_op</code> 为正表示 V
内核态 (<code>ipc/sem.c</code>)	<code>perform_atomic_semop()</code> 第一遍遍历检查 <code>semval >= sop </code> ，满足 → 第二遍执行 <code>semval -= sop </code> 。不满足 → 回滚已执行的所有操作 → <code>schedule_timeout()</code> 睡眠	<code>perform_atomic_semop()</code> 无条件 <code>semval += sop</code> → 遍历 <code>pending_alter</code> 队列 → 对每个等待者重试其操作集 → 成功则 <code>wake_up_process()</code> 唤醒

关键细节——内核如何保证操作数组的整体原子性。`semop()` 允许用户态一次提交多个操作 (`nsops` 参数)，这些操作必须全部成功或全部不执行。内核的 `perform_atomic_semop()` 采用类似银行家算法的”试分配”策略：

- 1. 第一遍——纯检查：遍历 `semop` 数组。对每个 V 操作 (`sop > 0`)，无条件视为可行。对每个 P 操作 (`sop < 0`)，检查 `semval >= |sop|`。记录每个操作需要修改的量，但不实际修改 `semval`。
- 2. 第二遍——执行 + 回滚：若第一遍全部通过，遍历数组执行修改：`V → semval += sop, P → semval -= |sop|`。若任一 P 操作不满足条件，回滚所有已执行的修改（那些 V 操作的 `semval` 增量要减回去），然后整个操作集标记为”阻塞”，进程进入 `TASK_INTERRUPTIBLE` 睡眠。
- 3. 唤醒后重试：当其他进程执行 V 操作后，内核遍历 `sma->pending_alter` 链表，对每个等待的进程从头重新执行完整的检查-执行流程。这意味着一个进程在睡眠期间，它的”所需资源数”没有锁住任何资源——其他进程可以继续消费信号量，被唤醒的进程重新参与竞争。

这套机制就是银行家算法在信号量领域的直接投影：银行家的安全序列检查 = `perform_atomic_semop` 的试分配 + 回滚；进程的 `Need` = `sembuf` 数组中的资源需求量；银

行家的 Available = 信号量当前值 `semval`。区别在于信号量不需要预先声明最大需求——`semop` 数组本身就是“这次要多少”，不满足就睡等下次。

实验 4 的 `sem_p/sem_v` 每次只操作一个信号量 (`nsops=1`)，所以原子性保证仅体现在单变量上。实验 8 的死锁检测则直接修改了内核中同一个 `ipc/sem.c`，在 `semctl()` 中添加了遍历 `pending_alter` 队列的逻辑——与 `perform_atomic_semop` 唤醒等待者时遍历的队列是同一个数据结构。

3.3 共享内存 (Shared Memory)

共享内存是最高效的 IPC 机制，因为数据传输完全绕过了内核缓冲区（零拷贝）。其实现充分利用了 Linux 虚拟内存管理的惰性分配和缺页处理能力。

创建阶段：`shmget()` → `newseg()` 在内核中分配 `struct shmid_kernel` 描述符，调用 `shmem_alloc_folio()` 从伙伴系统获取物理页框。但此时这些页框尚未映射到任何进程的地址空间——它们仅存在于内核的 `page cache` 中，等待后续缺页异常完成实际映射。

附加阶段：`shmat()` → `do_shmat()` 调用 `do_mmap()` 在进程地址空间中创建 VMA。此处关键设计在于 VMA 的 `vm_ops` 被设置为 `shmem_vm_ops`，其中 `.fault = shmem_fault`。这意味着进程首次访问共享内存地址时触发缺页异常，`handle_mm_fault()` → `do_fault()` → `shmem_fault()` 将之前分配的物理页框映射到进程页表中。这一惰性映射机制确保了即使分配了大量共享内存，未实际使用的页面不会消耗 TLB 条目。

分离阶段：`shmdt()` 仅从进程地址空间中解除 VMA 映射，不释放底层物理内存。物理页框持续存在直到显式调用 `shmctl(IPC_RMID)` 删除共享内存段。这种“映射可断、数据不丢”的设计允许多个进程在各自的时间窗口内独立附加和分离同一共享内存段，非常适合长期运行的数据交换场景。

3.4 信号 (Signal)

发送路径：`send_signal()` → `__send_signal_locked()` (`kernel/signal.c`)，将 `struct sigqueue` 加入目标进程 `pending` 队列，调用 `complete_signal()` → `signal_wake_up()` 设置 `TIF_SIGPENDING` 并唤醒进程。

处理路径：进程从内核态返回用户态时检查 `TIF_SIGPENDING`，调用 `arch_do_signal_or_restart()` (`arch/x86/kernel/signal.c`) → `handle_signal()`，在用户栈上构建信号帧 (`setup_rt_frame()`)，修改用户态 RIP 指向 handler 入口。handler 返回后通过 `sigreturn()` 恢复原执行上下文。

信号交付本质上是一次“同进程内的用户态 mini context switch”。这个过程需要在内核态和用户态之间做一次精密的控制转移，其复杂性不亚于调度器中的上下文切换，只是范围局限在单个进程内部。完整时序如下：

1. **保存现场**：内核在进程的用户栈上构造 `struct rt_sigframe`，其中包含被打断时的全部通用寄存器 (`RAX/RBX/RCX/.../R15`)、RIP (被打断的指令地址)、RSP (被打断时的用户栈指针)、RFLAGS，以及信号编号和附加信息 (`siginfo_t`)。
2. **注入返回地址**：在信号帧中嵌入一个伪造的返回地址——它指向 `vDSO` 中的 `__kernel_rt_sigreturn` 入口。这不是普通的函数返回地址，而是一个“陷阱”——handler 正常 `ret` 时就会落入这个地址，触发 `sigreturn()` 系统调用。
3. **劫持控制流**：修改 `pt_regs` 中的 RIP 指向信号 handler 的第一条指令，修改 RSP 指向信号帧上方 (handler 使用的新栈区域)。由于这些修改发生在内核态的 `pt_regs` 结构上，当进程通过

`sysretq` 返回用户态时，硬件自动从 `pt_regs` 恢复 RIP/RSP——“进程”看起来“像是在 handler 入口处开始执行”。

4. **handler 执行**: handler 在用户态运行，使用内核为其铺设的新栈区域。它可以访问 `siginfo_t` 和 `ucontext_t`（通过 handler 参数传入，指向信号帧中的对应区域）。
5. **自动折返**: handler 执行 `ret` 指令。由于返回地址是 `__kernel_rt_sigreturn`，CPU 跳转到 vDSO 中的 `syscall` 指令 → 进入内核 `sigreturn()` 系统调用。内核从用户栈上保存的 `rt_sigframe` 中恢复全部通用寄存器、RIP、RSP、RFLAGS → `sysretq` → 进程回到被打断的原始位置，仿佛什么都没发生过。

这个过程和调度器上下文切换的对称性值得注意：调度器的 `__switch_to_asm` 在**内核栈**上保存/恢复寄存器，切换的是**两个不同进程**的执行流；信号交付在**用户栈**上保存/恢复寄存器，切换的是**同一进程的两个用户态上下文**（主执行流和信号 handler）。两者都依赖“在某处保存全量寄存器状态 → 修改 RIP → 未来从保存点恢复”的机制，只是载体不同。

3.5 管道 (Pipe)

管道(`fs/pipe.c`)是 Unix 最经典的 IPC 机制，实现为内核中的环形缓冲区。默认容量为 16 个页 (64KB)，可通过 `fcntl(F_SETPIPE_SZ)` 调整。管道的数据结构 `struct pipe_inode_info` 包含一个 `struct pipe_buffer` 数组（环形缓冲区的槽位）和读写指针。

`pipe_write()` 的调用链为 `ksys_write()` → `vfs_write()` → `pipe_write()`。写入操作首先检查管道是否已有读端关闭（若关闭则发送 `SIGPIPE` 信号），然后通过 `pipe_get_tx_ring_size()` 获取可用空间。若管道有足够空间容纳写入数据，`copy_page_from_iter()` 将用户空间数据复制到内核中的 pipe buffer 页面；若管道满，根据 `O_NONBLOCK` 标志决定是否阻塞——阻塞模式下写入进程进入 `TASK_INTERRUPTIBLE` 睡眠，等待读端消费数据后通过 `poll_wake()` 唤醒。

`pipe_read()` 执行对称操作：检查管道是否有数据，若有则通过 `copy_page_to_iter()` 将内核 pipe buffer 数据复制到用户空间缓冲区，更新读指针；若管道空则根据 `O_NONBLOCK` 决定阻塞或立即返回 `-EAGAIN`。

管道的核心设计限制是**半双工**——数据只能单向流动。若需要双向通信，通常创建一对管道或使用 Unix Domain Socket。管道的最大优势在于其简单性和与 shell 管线的无缝集成（`pipe()` + `fork()` + `dup2()` + `exec()` 的经典模式），使得进程间数据流式处理无需任何显式的同步原语——管道的 FIFO 顺序和阻塞语义天然提供了生产者-消费者同步。

3.6 实验验证

3.6.1 实验 4：生产者-消费者多进程实现

本实验使用 System V IPC 实现了 3 生产者 + 4 消费者的经典多进程同步模型。核心设计：共享环形缓冲区（容量 8）通过 XSI 信号量集的三个信号量保护——`MUTEX`（互斥锁，初值 1）、`EMPTY`（空槽计数，初值 8）、`FULL`（满槽计数，初值 0）。生产者流程为 `P(EMPTY)` → `P(MUTEX)` → 写入 → `V(MUTEX)` → `V(FULL)`，消费者流程为 `P(FULL)` → `P(MUTEX)` → 读取 → `V(MUTEX)` → `V(EMPTY)`。共享缓冲区通过 `shmget()` + `shmat()` 映射到各进程地址空间。

关键技术细节：`fork()` 之前调用 `setvbuf(stdout, NULL, _IONBF, 0)` 禁用 `stdio` 缓冲，避免 `fork` 后子进程复制缓冲区导致输出重复；消费者退出前执行级联唤醒——检测到总消费量已达目标后额外执行一次 `V(FULL)`，防止最后一个消费者阻塞在 `P(FULL)` 上无人唤醒。

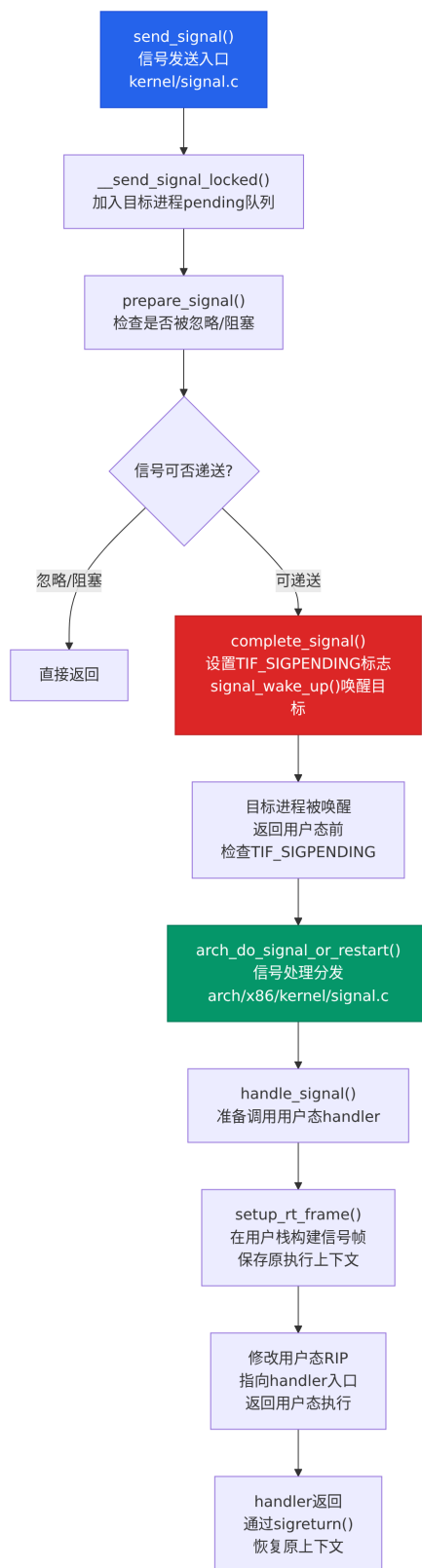


图 7: Linux 信号发送与处理流程

验证结果：3 生产者各生产 5 项（总计 15），4 消费者共消费 15 项，缓冲区残留为 0，所有 7 个子进程正常退出。多次运行结果一致，确认无死锁、无活锁、无数据竞争。该实验直接验证了 §3.2 所述的信号量 P/V 原子操作——`semop()` 系统调用进入 `ipc/sem.c` 中的 `perform_atomic_semop()`，通过原子操作集语义保证了要么全部成功要么全部回滚。

3.6.2 实验 8：信号量死锁检测与解除

本实验对内核 `ipc/sem.c` 进行了功能性扩展，添加了两个自定义 `semctl` 命令。`detect_sem_deadlock()` (`DEADCHECK = 0xDEAD`) 的检测逻辑为：遍历信号量集中的全部信号量，若每个 `semval` 均为 0 且阻塞等待进程数 `2` 且 信号量数，则判定为潜在死锁。`break_sem_deadlock()` (`DEADBREAK = 0xBEAC`) 选择等待队列中 RSS 最小的进程（作为“代价最小”的受害者），通过 `kill_pid(task_pid(victim), SIGKILL, 1)` 强制终止以打破环路。

开发过程中修复了两个关键 bug：一是最初只扫描全局等待队列 `sma->pending_alter`，遗漏了 per-信号量等待队列 `sma->sems[i].pending_alter`；二是 glibc 的 `semctl()` 包装函数在用户态检查 `cmd` 参数并拒绝未知值——`semctl(semid, 0, 0xDEAD)` 根本未进入内核，需改用 `syscall(__NR_semctl, ...)` 直接调用。

验证测试构造了经典的 AB-BA 死锁（P1 持有 `sem[0]` 等待 `sem[1]`，P2 持有 `sem[1]` 等待 `sem[0]`），`DEADCHECK` 成功返回 1（检测到死锁），`DEADBREAK` 成功返回 0 并终止 RSS 较小的进程。内核日志输出 “[sem_deadlock] DEADLOCK DETECTED! semid=65538 waiters=2 nsems=2”。该实验验证了死锁检测算法在真实内核 IPC 子系统实现中的可行性。

4 物理内存管理和分配机制

4.1 NUMA 与内存节点

在 NUMA 架构下，物理内存按节点（node）、区域（zone）两级组织 [12]。NUMA 的核心思想是“本地内存访问快、远端内存访问慢”——每个 CPU 直接连接一部分物理内存（本地节点），访问该节点的内存延迟最低；跨节点访问需要经过 CPU 间互联总线，延迟显著增加（在大型服务器上可达 1.5–3 倍）。

每个节点由 `pglist_data` 结构描述（`include/linux/mmzone.h`）。该结构包含节点 ID、节点起始页框号 `node_start_pfn`、节点拥有的物理页数 `node_present_pages`、以及该节点上的所有 zone 数组 `node_zones[MAX_NR_ZONES]`。关键的 `node_zonelists` 定义了该节点进行内存分配时的 zone 遍历顺序——优先从本节点的 `ZONE_NORMAL` 分配，若不足再尝试本节点的 `ZONE_DMA32`，最后才回退到远端节点的内存。这一“本地优先、远端兜底”的策略在整个伙伴系统的 `get_page_from_freelist()` 中通过遍历 `zonelist` 自动贯彻。

在 x86_64 架构下，每个节点内部划分四个区域：`ZONE_DMA`（0–16MB，传统 ISA 设备 DMA 需要物理地址在此范围）、`ZONE_DMA32`（16MB–4GB，32 位 PCI 设备 DMA 限制）、`ZONE_NORMAL`（4GB 以上至内核直接映射上限 `PAGE_OFFSET + 64TB`，内核通过 `__va()/__pa()` 宏实现虚拟地址与物理地址的 O(1) 转换）、`ZONE_MOVABLE`（用于内存热插拔和碎片整理的可迁移页面区域）。区域划分使得内核在分配时可以根据请求的 `gfp` 标志（如 `GFP_DMA`，`GFP_DMA32`，`GFP_HIGHUSER_MOVABLE`）精确选择满足硬件约束的物理内存。

4.2 伙伴系统 (Buddy System)

伙伴系统是 Linux 物理页分配的基础算法 (mm/page_alloc.c), 管理 $\text{MAX_ORDER} - 1 = 10$ 个阶 (order 0–10, 即 1 页至 1024 页连续块) [12, 5]。

4.2.1 核心数据结构

```
1 struct free_area {
2     struct list_head free_list[MIGRATE_TYPES]; // 按迁移类型分类
3     unsigned long nr_free;                       // 该阶空闲块数量
4 };
5 struct zone {
6     struct free_area free_area[MAX_ORDER]; // 各阶空闲链表
7     unsigned long managed_pages;
8     // ...
9 };
```

Listing 36: free_area —伙伴系统空闲链表 (include/linux/mmzone.h)

4.2.2 分配路径详解

页分配的主入口为 `alloc_pages()`, 最终进入 `__alloc_pages_noprof()` (mm/page_alloc.c)。该函数首先通过 `prepare_alloc_pages()` 设置分配上下文 (`gfp_t` 标志位决定分配行为: `GFP_KERNEL` 允许睡眠和回收、`GFP_ATOMIC` 禁止睡眠、`__GFP_HIGH` 允许使用紧急保留内存等)。

随后进入**快速路径**——`get_page_from_freelist()`: 该函数遍历 `zonelist` (区域优先级列表), 在每个 `zone` 中查找 `free_area[order]`。若请求的 `order` 有空闲块, 调用 `__rmqueue()` 从链表中取出。`__rmqueue()` 的关键优化是先在新增的 `MIGRATE` 类型 (如 `MIGRATE_UNMOVABLE`, `MIGRATE_RECLAIMABLE`, `MIGRATE_MOVABLE`) 对应链表中查找, 这一机制通过将不同生命周期的页面分类存储来减少内存碎片。若请求阶数无空闲块, 调用 `__rmqueue_fallback()` 向高阶借块——将 2^{k+1} 的伙伴块按地址一分为二, 低地址半部用于分配, 高地址半部作为 2^k 阶块归还低阶链表 (`expand()` 函数执行此分裂)。

若快速路径失败, 进入**慢速路径**——`__alloc_pages_slowpath()`: 首先尝试异步内存压缩 (`should_compact_retry()` + `try_to_compact_pages()`), 将分散的空闲页聚集为高阶连续块; 若仍不足, 唤醒 `kswapd` 执行页面回收, 将 `Inactive` 链表中的页面换出以释放物理页。经过多轮重试 (最多 `MAX_RECLAIM_RETRIES` 次), 若所有努力均无法满足分配请求且 `gfp_t` 中设置了 `__GFP_RETRY_MAYFAIL` 以外的标志, 则 OOM killer 选择并终止内存消耗最大的进程。

4.2.3 释放路径详解

`__free_pages()` 是页释放的主入口。释放路径的精妙之处在于**伙伴合并**: 函数 `__free_one_page()` 将待释放页框加入对应 `order` 的空闲链表, 然后调用 `__find_buddy_pfn()` 计算伙伴页框号。在伙伴系统中, 两个页框互为伙伴当且仅当: 它们的页框号仅在第 `order` 位不同 (即 `pfn ^ (1 << order)`)。通过简单的位运算即可快速定位伙伴, 无需遍历链表。

若伙伴同阶且空闲——即满足 `page_is_buddy()` 检查 (伙伴不在使用中、同阶、同 `zone`), 则将两个 `order-k` 伙伴从空闲链表中移除, 合并为一个 `order-(k+1)` 块, 然后递归尝试与新的伙伴合并。这一过程持续到无法再合并为止——此时所有可合并的伙伴均已聚合为最大连续块, 最大化了大块连续物理内存的可用性。

伙伴系统的根本缺陷与 MIGRATE_TYPES 的补救。理论上伙伴系统的外部碎片率不超过 33%，但实践中远不止此。问题出在页面生命周期的异质性：内核通过 GFP_KERNEL 分配的页面（如 kmalloc 的对象、页表页）在整个系统运行期间散布在各处且无法迁移。当一个不可移动页恰好落在两个可合并的伙伴之间时，合并链被切断——该页两侧的大块永远无法聚合，即使系统有大量空闲内存也凑不出高阶连续块。Linux 通过 MIGRATE_TYPES 机制缓解此问题：将空闲页面按迁移属性分为 MIGRATE_UNMOVABLE（内核核心分配，不可移动）、MIGRATE_RECLAIMABLE（可回收，如 inode/dentry cache）、MIGRATE_MOVABLE（用户态页面，可通过缺页异常迁移）、MIGRATE_CMA（连续内存分配器专用）等类型。__rmqueue() 分配时优先在匹配的迁移类型链表中查找，同类页面聚集在一起，碎片只发生在同类型内。但还有一个更深的问题：MIGRATE_UNMOVABLE 区域会随时间逐渐扩张（内核分配无处不在），侵蚀可移动区域。memory compaction (mm/compaction.c) 正是为此存在——它主动迁移 MIGRATE_MOVABLE 页面到其他位置，为不可移动分配腾出连续空间。实验 10 的 page_stats.ko 逐页遍历时通过 PageBuddy/PageSlab/PageLRU 等宏分类统计，正是对这几种迁移类型的间接观测。

图 8 展示了伙伴系统分配与释放的完整流程。

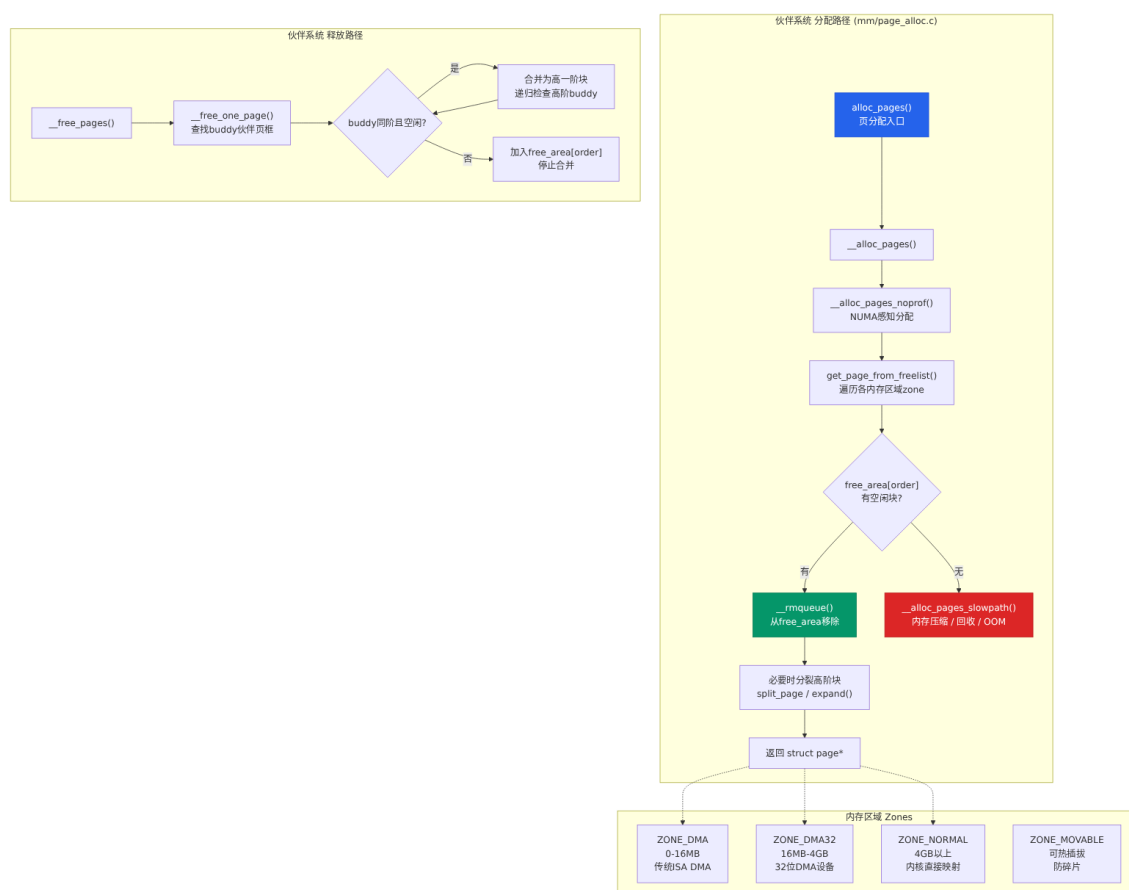


图 8: 伙伴系统页分配与释放流程

4.3 Slub 分配器

伙伴系统以页为分配粒度，对于内核中频繁使用的小对象（如 task_struct, inode, dentry），Linux 使用 slub 分配器 (mm/slub.c) [13]。每个 slab 缓存 (struct kmem_cache) 管理特定大小的对象，通过 kmem_cache_alloc() / kmem_cache_free() 进行快速分配和释放。slub 分配器从伙伴

系统获取大块页面，内部细分为等大小的对象槽位，利用 per-CPU 缓存实现无锁快速路径。

4.4 实验验证

4.4.1 实验 10：内存管理分析与验证（物理内存部分）

本实验通过三个用户态程序和一个内核模块对第 4-5 章的内存管理理论进行了系统验证。

`mem_observe.c` 通过 `strace -e trace=brk,mmap,munmap` 跟踪 `malloc()` 的底层行为，精确测定了 glibc 的分配策略阈值：64KB 时使用 `brk()` 扩展堆区（连续地址，无法单独释放中间块）；1MB 时使用 `mmap()` 匿名映射（独立 VMA，可独立 `munmap`）；过渡区间 64KB-1MB 根据 glibc 内部策略动态选择。该观察直接印证了 §4.2 和 §5.2 所述的两种内存分配路径。

`mem_alloc.c` 通过 `mmap` 20000 个 4KB 页面（共 80MB）对比两种场景验证惰性分配：只 `mmap` 不访问时 VIRT 增长 80MB 但 RES 几乎不变——页表项为空，未触发缺页；调用 `memset` 逐页写入后 RES 与 VIRT 同步增长至 80MB——每次首次写入触发一次缺页异常 → `do_anonymous_page()` 分配物理页框。该实验直观展示了虚拟内存惰性分配的“承诺 vs 兑现”机制。

`page_stats.ko` 内核模块遍历所有 NUMA 节点和内存区域，通过 `pfn_to_page()` 逐页统计使用 `PageBuddy/ PageSlab/ PageLRU/ folio_test_active/ PageTable/ PageReserved` 等宏分类。Linux 6.18 中需使用 `folio` API (`folio_test_active(page_folio(page))` 替代已废弃的 `PageActive()`)，且 `first_online_pgdat()/next_online_pgdat()` 未导出，需改用 `for_each_online_node(nid) + NODE_DATA(nid)` 遍历。模块输出与 `/proc/meminfo` 数据吻合，验证了伙伴系统各阶空闲块的分布与内核页面统计的一致性。

5 虚拟存储管理和换页机制

5.1 虚拟地址空间管理

进程的虚拟地址空间由 `struct mm_struct` (`include/linux/mm_types.h`) 描述 [9, 12]。其中包含 VMA 链表 (`mmap`) 和红黑树 (`mm_rb`) 用于快速查找地址对应的 VMA，以及页全局目录指针 (`pgd`) 指向进程页表根。

5.2 `vm_area_struct` (VMA)

每个 VMA 描述进程地址空间中一段连续的虚拟地址区间：

```
1 struct vm_area_struct {
2     unsigned long vm_start;           // 区间起始虚拟地址
3     unsigned long vm_end;             // 区间结束虚拟地址
4     pgprot_t vm_page_prot;            // 页保护属性
5     unsigned long vm_flags;           // VM_READ|VM_WRITE|VM_EXEC|VM_SHARED...
6     const struct vm_operations_struct *vm_ops; // VMA 操作回调
7     unsigned long vm_pgoff;           // 文件映射偏移（页单位）
8     struct file *vm_file;             // 文件映射的 struct file
9     struct mm_struct *vm_mm;          // 所属 mm_struct
10    struct rb_node vm_rb;              // 红黑树节点
11 };
```

Listing 37: `vm_area_struct` 核心字段 (`include/linux/mm_types.h`)

5.3 mmap 系统调用与惰性分配

`mmap()` → `do_mmap()` (`mm/mmap.c`) 是创建虚拟内存映射的核心入口 [23]。`mmap_region()` 分配 `vm_area_struct` 并设置地址范围、权限、文件映射信息，通过 `vma_link()` 插入 `mm->mmap` 链表和 `mm->mm_rb` 红黑树。

关键设计——惰性分配：`mmap()` 仅创建 VMA 记录映射关系，不立即分配物理页。进程首次访问该地址时触发缺页异常，由 `handle_mm_fault()` 完成页表项的实际填充。这极大节省了物理内存，如图 9 所示。

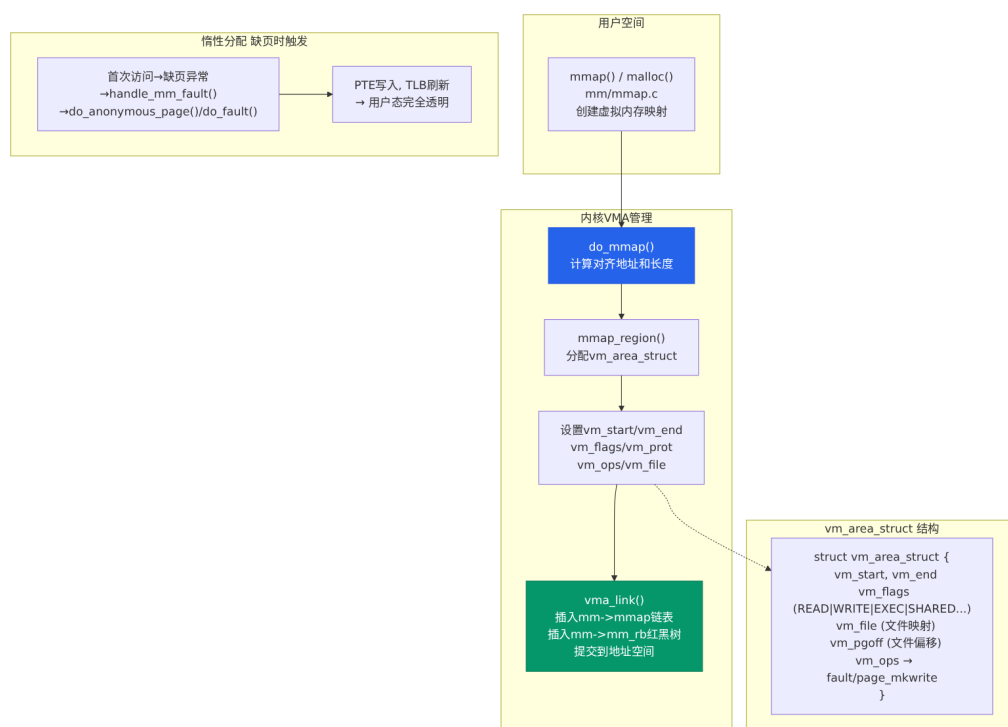


图 9: mmap 系统调用与 VMA 惰性分配机制

5.4 缺页异常处理

x86_64 下缺页异常由 IDT 表项 `exc_page_fault` 调用 `handle_page_fault()` (`arch/x86/mm/fault.c`)，从 CR2 获取 `fault` 地址、解析 `error_code`，分发至 `handle_mm_fault()` (`mm/memory.c`) [25]。

`handle_mm_fault()` 完成五级页表遍历 (`pgd`→`p4d`→`pud`→`pmd`→`pte`)，根据 PTE 状态分发。值得深入分析的是各级页目录的**按需分配**机制：内核在遍历过程中可能发现某级页目录不存在（如 `pgd` 条目为空），此时调用 `p4d_alloc()`、`pud_alloc()`、`pmd_alloc()` 逐级分配中间页表。这一懒分配策略与 VMA 的惰性分配一脉相承——只有真正需要写入页表条目时才分配中间页目录的内存。分配完所有中间目录后，到达最终 PTE 层，根据具体状态走四种路径之一：

- **PTE 为空（首次访问）：**`do_anonymous_page()` 处理匿名映射的首次访问。对于读访问，分配一个全零的“零页”（`ZERO_PAGE`）并映射为只读——所有仅读取而未写入的匿名页共享同一个物理零页，节省内存。对于写访问，通过伙伴系统分配新物理页框并清零（防止信息泄露），设置 PTE 为可写，完成从虚拟地址到物理页框的绑定。若此匿名页由 `fork()` 的 COW 产生，则标记为写保护。

- **PTE 为 swap 条目**: `do_swap_page()` 处理被换出页面的重新访问。swap 条目中编码了 swap 设备号和槽位偏移，内核通过 `lookup_swap_cache()` 或 `read_swap_cache_async()` 从 swap 设备读回页面数据。若该页面同时存在于 swap cache（其他进程恰好也在换入同一页），则直接共享 page cache 中的页面，避免重复 I/O。
- **PTE 为文件页**: `do_fault()` 处理文件映射（mmap 的文件页或 ELF 可执行文件的代码段）。数据流为: `do_fault()` → `do_read_fault()` → `__do_fault()` → `vmf->vma->vm_ops->fault()`。以 ext2 文件映射为例，最终调用 `filemap_fault()`，在 page cache 中查找文件偏移对应的 page，若未命中则调用 `ext2_read_folio()` 从磁盘读取数据填充新 page。
- **PTE 存在但写保护 (COW)**: `do_wp_page()` 处理写时复制。这是 fork 后父子进程共享页面的关键技术，详见下一小节的深入分析。

图 10 展示了缺页异常处理的完整决策树。

5.5 Copy-on-Write (COW) 深入分析

COW 是 Linux 虚拟内存管理的核心优化 [9]。在 `fork()` 期间，`copy_mm()` → `dup_mmap()` 复制父进程的所有 VMA，将父子进程的页表项均标记为只读并设置 `PTE_SOFT_DIRTY`。任一进程尝试写入共享页时触发写保护缺页，`do_wp_page()` (`mm/memory.c`) 执行：

```
1 static vm_fault_t do_wp_page(struct vm_fault *vmf) {
2     // 1. 获取缺页所在的 VMA 和页表项
3     vma = vmf->vma;
4     // 2. 获取物理页 folio
5     folio = page_folio(vmf->page);
6     // 3. 关键判断: folio 的引用计数
7     if (folio_likely_mapped_shared(folio)) {
8         // 多个进程共享 → 必须分配新页、复制内容
9         new_folio = folio_copy(folio, vma, vmf->address);
10        page_add_new_anon_rmap(new_folio, vma, vmf->address);
11    } else {
12        // 仅一个使用者 → 直接在原页上解除写保护
13        wp_page_reuse(vmf);
14        folio_mark_accessed(folio);
15    }
16    // 4. 更新页表项为可写，刷新 TLB
17 }
```

Listing 38: `do_wp_page` 核心逻辑 (`mm/memory.c`)

关键优化：若 `folio_ref_count == 1`（已无其他使用者，如父进程退出后子进程独占），直接在原页上解除写保护而无需复制，避免了不必要的内存分配和拷贝开销。

COW 的工程经济学：为什么 fork+exec 组合几乎免费。COW 最重要的设计动机不是节省 fork 后的写入——而是让 fork 后立即 exec 的场景不浪费任何物理内存。考虑 shell 执行 `ls` 的典型流程：

- `bash` (200MB 常驻内存) 调用 `fork()`。不用 COW 的话，需要立即分配 200MB 物理内存并逐页复制内容。然后 `exec("ls")` 调用 `flush_old_exec()` → `exec_mmap()` 释放整个地址空间——200MB 的分配和复制全部作废。

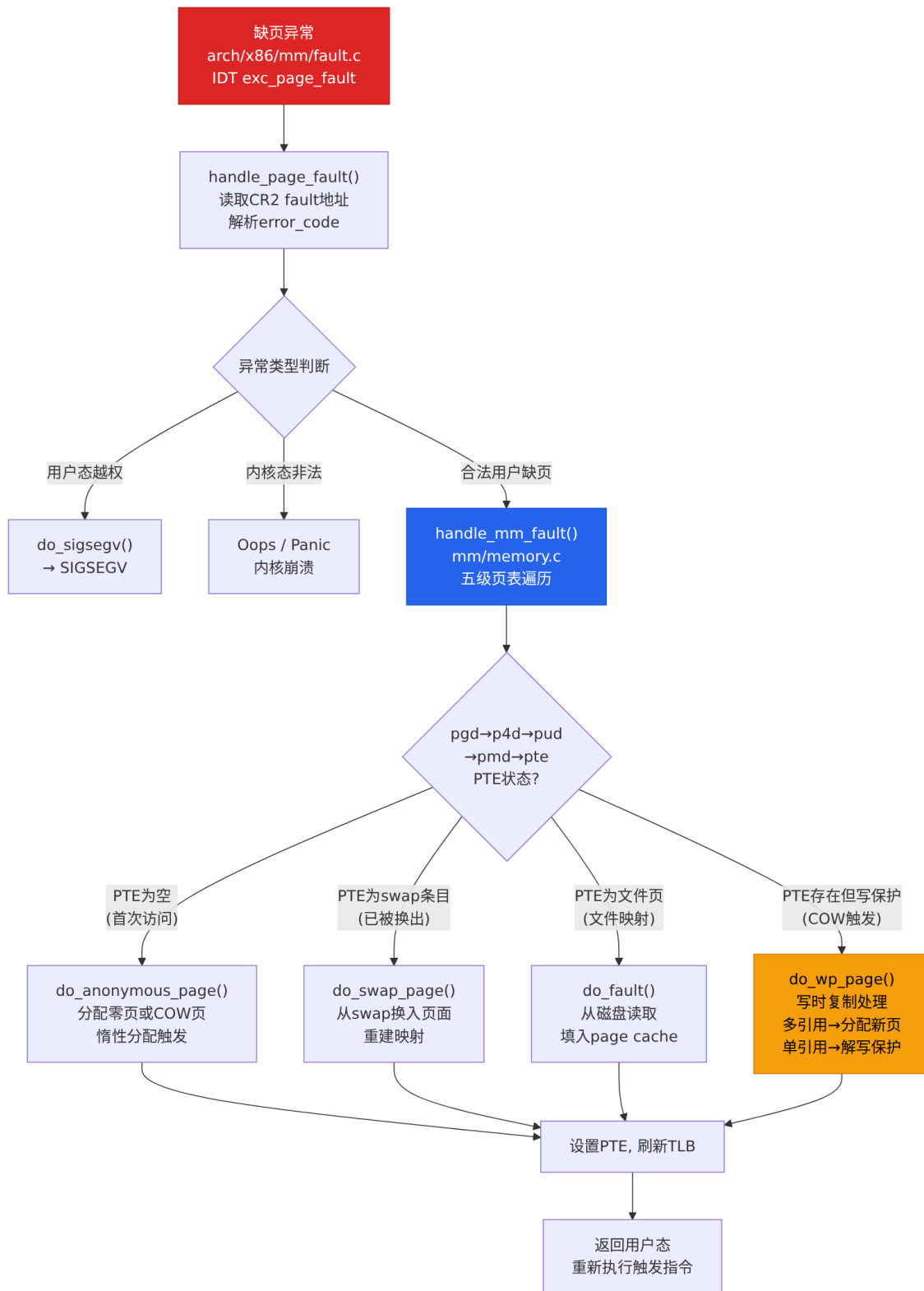


图 10: 缺页异常处理完整流程

- 用 COW: `fork()` 只复制页表。bash 的 200MB 地址空间按 4KB 分页, 共约 51,200 个 PTE, 每个 PTE 8 字节, 页表复制开销约 400KB。父子进程的页表项均标记为只读 (PTE_RDONLY | PTE_SOFT_DIRTY), 但物理页框不做任何复制——父子共享全部 200MB 物理页。
- `exec("ls")` 调用 `exec_mmap()` → `mmap(old_mm)`, 旧 `mm_struct` 的引用计数递减。页表被释放 (400KB 回收), 但物理页框因为 bash 父进程仍持有引用而不释放。子进程从未写入过任何共享页——零 COW 缺页发生。
- 总开销: 400KB 的页表复制 + 释放。与父进程内存大小无关。

这个设计使得 Unix 的“fork+exec 创建新进程”模式在工程上可行——否则每次敲命令都要复制整个 shell 的内存, 200MB 的 bash 会让系统寸步难行。实验 10 的 `mem_cow.c` 通过 `/proc/<pid>/smaps` 观察 PSS (Proportional Set Size) 值验证了此机制: fork 后父子 PSS 各为原值的一半 (共享页按引用计数均分), 子进程写入后才各自增长为独立物理页的完整值。

当系统内存压力增大时, `kswapd` 守护线程被唤醒执行页面回收 (`mm/vmscan.c`)。回收策略: 遍历 LRU 链表 (Active/Inactive × Anonymous/File 四个链表), 优先回收 Inactive 端页面。匿名页通过 `add_to_swap()` 分配 swap 槽位写入 swap 设备, PTE 替换为 swap 条目。文件页若干净可直接丢弃 (可从磁盘重读), 若脏页先调用 `writepage()` 写回。

5.6 实验验证

5.6.1 实验 10: 内存管理分析与验证 (虚拟内存部分)

`mem_cow.c` 是验证写时复制机制的核心测试程序。父进程通过 `mmap(MAP_PRIVATE|MAP_ANONYMOUS|MAP_POPULATE, size)` 分配 N GB 内存并 `memset` 写入初始值 (强制触发所有页面的缺页分配), 然后 `fork()`。fork 后通过 `top` 观察: 父子进程 VIRT 各显示 N GB, 但系统总 RES 仅增长至 N GB (而非 2N GB) ——父子共享相同的物理页框, 页表项均标记为只读。子进程随后 `memset` 写入前一半页面, 触发 `do_wp_page()` 为每个被写入的页面分配新的物理页并复制内容, 系统总 RES 增长至 1.5N GB。通过 `/proc/<pid>/smaps` 可逐 VMA 确认 COW 标记 (`VmFlags: ... ac`) 和实际物理内存占用 (PSS 值)。该实验精确验证了 §5.4 所述的 COW 引用计数判断和惰性复制机制。

5.6.2 实验 5: 目录递归拷贝的单/多进程实现

本实验实现了两个文件操作程序, 从实践层面验证了 VFS 层和进程创建的理论认知。

`mycp` (单进程递归拷贝) 使用 POSIX 标准接口实现文件复制: `open(O_RDONLY) + fstat() + open(O_WRONLY|O_CREAT|O_TRUNC)` 分别打开源文件和目标文件, 通过 8KB 缓冲区的 `read/write` 循环搬运数据, 内层 `while (remaining > 0)` 保证处理 `write` 的部分写入场景 (`write` 返回值可能小于请求量)。目录递归通过 `opendir/readdir/lstat` 遍历 (`lstat` 而非 `stat` 以避免跟踪符号链接到达循环目标), 符号链接通过 `readlink + symlink` 复制链接本身而非目标。文件时间戳通过 `utimensat` 保留。所有场景 (文件 → 文件、文件 → 目录、目录 → 目录、符号链接、目标已存在) 均通过 `diff -r` 验证全结构等价。

`myls` (多进程并发拷贝) 通过 `opendir/readdir` 遍历源目录树, 对每个普通文件执行 `fork() + execlp("./mycp", ...)` 启动子进程拷贝, 滑动窗口机制将最大并发子进程数限制为 16。所有子进程通过 `wait()` 回收, `diff -r` 验证结果与单进程版本完全一致, 且无僵尸进程 (PID 泄漏)。

该实验从用户态验证了 `open()/read()/write()` 系统调用的完整路径：用户调用 → VFS 层的 `sys_openat()` → `do_filp_open()` → `path_openat()` → `link_path_walk()`（路径解析）→ `do_dentry_open()`（设置 `file->f_op`），以及 `read()` → `vfs_read()` → `f_op->read_iter` → `generic_file_read_iter()` → `filemap_read()` → `do_fault()` 将磁盘数据读入 page cache 的完整数据流。`fork()` + `exec()` 的组合验证了进程创建（COW 地址空间复制）和镜像替换（完全替换地址空间的代码、数据、堆、栈）的完整生命周期。

6 虚拟文件系统和 ext2 文件系统实现

6.1 VFS 虚拟文件系统概述

VFS (Virtual File System) 是 Linux 内核中抽象文件系统操作的统一接口层 [6, 14]。其设计灵感源于 Unix 的“一切皆文件”哲学——无论是磁盘上的 ext2 文件、`procfs` 中的进程信息、还是设备文件 `/dev/sda`，内核均通过相同的系统调用接口（`open/read/write/close`）提供服务。VFS 通过四个核心对象模型实现这一抽象：

super_block (`include/linux/fs.h`): 代表一个已挂载的文件系统实例。每个 `mount` 操作创建一个 `super_block`，其中 `s_op` 指向文件系统特定的 `super_operations`（如 `ext2_write_inode` 将 `inode` 变化写回磁盘，`ext2_put_super` 在卸载时释放资源），`s_root` 指向该文件系统根目录的 `dentry`。

inode: 文件系统中每个文件或目录在内存中的唯一表示。`inode` 携带文件的全部元数据（权限、大小、时间戳）和操作表 `i_op` (`inode_operations`: `create`, `lookup`, `link`, `unlink`, `mkdir`, `rmdir`, `rename` 等目录操作) 及 `i_fop` (`file_operations`: `read_iter`, `write_iter`, `mmap`, `fsync` 等文件内容操作)。

dentry (Directory Entry): 路径名解析的缓存。每个路径分量对应一个 `dentry`，通过其父 `dentry` 的 `d_children` 链表和全局 `d_hash` 哈希表双向索引。`dcache` 的精妙之处在于其状态机：`dentry` 可处于 `unused`（无 `inode` 关联但保留在 `dcache` 中以备后续查找加速）、`in-use`（有 `inode` 关联且被引用）、`negative`（对应不存在的文件路径，缓存“查找失败”结果）三种状态。

file: 进程打开文件后的上下文对象。每个 `open()` 调用创建一个 `struct file`，其中 `f_op` 指向该文件类型的操作表（通常从 `inode->i_fop` 复制），`f_pos` 跟踪当前读写偏移，`f_flags` 记录 `O_RDONLY/O_WRONLY/O_APPEND` 等打开标志。

图 11 展示了 VFS 核心对象及其与具体文件系统的关系。

6.2 路径名解析—`link_path_walk()`

`sys_open()` → `path_openat()` (`fs/namei.c`) 是文件打开的主调用链。核心路径解析由 `link_path_walk()` 完成：

```
1 static int link_path_walk(const char *name, struct nameidata *nd) {
2     for (;;) {
3         // 1. 跳过连续的 '/'
4         while (*name == '/') name++;
5         // 2. 提取下一路径分量
6         hash = init_name_hash(nd);
7         do { hash = partial_name_hash(c, hash); } while (...);
8         // 3. 在 dcache 中查找 dentry (快速路径)
9         dentry = walk_component(nd, WALK_NOFOLLOW);
10        // 4. 若 dcache 未命中 → 调用 i_op->lookup() 从磁盘读取
```

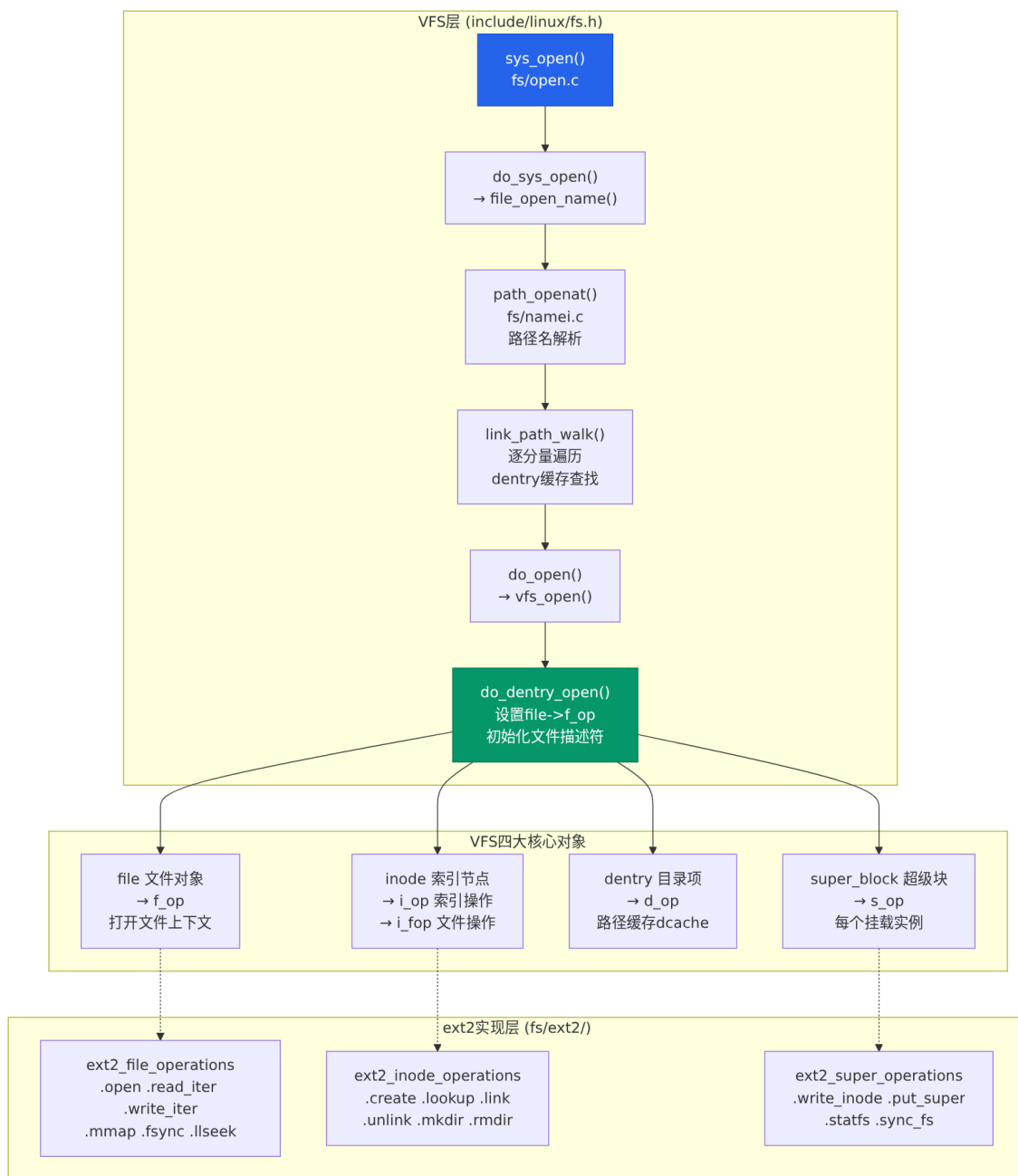


图 11: VFS 核心对象模型与 ext2 实现层

```

11     if (IS_ERR(dentry)) break;
12     // 5. 处理符号链接、挂载点穿越
13     if (d_is_symlink(dentry)) { /* ... */ }
14 }
15 }

```

Listing 39: link_path_walk 路径解析 (fs/namei.c)

walk_component() 首先在 nd->path.dentry 的 dcache 子节点中查找 (哈希 + DRCU 锁保护)。若命中 (大多数情况), O(1) 返回 dentry。若未命中, 调用父目录 inode 的 i_op->lookup() (如 ext2 的 ext2_lookup()), 从磁盘读取目录项并创建新 dentry 加入 dcache。

6.3 文件操作—read_iter / write_iter

现代 Linux 文件 I/O 通过 read_iter / write_iter 方法完成 [10]。这些方法接收 struct kiocb (内核 I/O 控制块) 和 struct iov_iter (用户空间缓冲区迭代器):

```

1 struct file_operations {
2     struct module *owner;
3     loff_t (*llseek)(struct file *, loff_t, int);
4     ssize_t (*read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*write)(struct file *, const char __user *, size_t, loff_t *);
6     ssize_t (*read_iter)(struct kiocb *, struct iov_iter *);
7     ssize_t (*write_iter)(struct kiocb *, struct iov_iter *);
8     int (*mmap)(struct file *, struct vm_area_struct *);
9     int (*open)(struct inode *, struct file *);
10    int (*fsync)(struct file *, loff_t, loff_t, int datasync);
11 };

```

Listing 40: file_operations 结构 (include/linux/fs.h)

对于 generic_file_read_iter() (mm/filemap.c), 数据流为: page cache 查找 → 若未命中分配新页 → 调用 ext2_read_folio() 从磁盘填充 → copy_page_to_iter() 复制到用户缓冲区。这是典型的”page cache → 磁盘 → 用户态”三层数据流。

6.4 ext2 文件系统实现

ext2 是 Linux 经典的磁盘文件系统 [29, 7]。图 12 展示了 ext2 的磁盘布局。

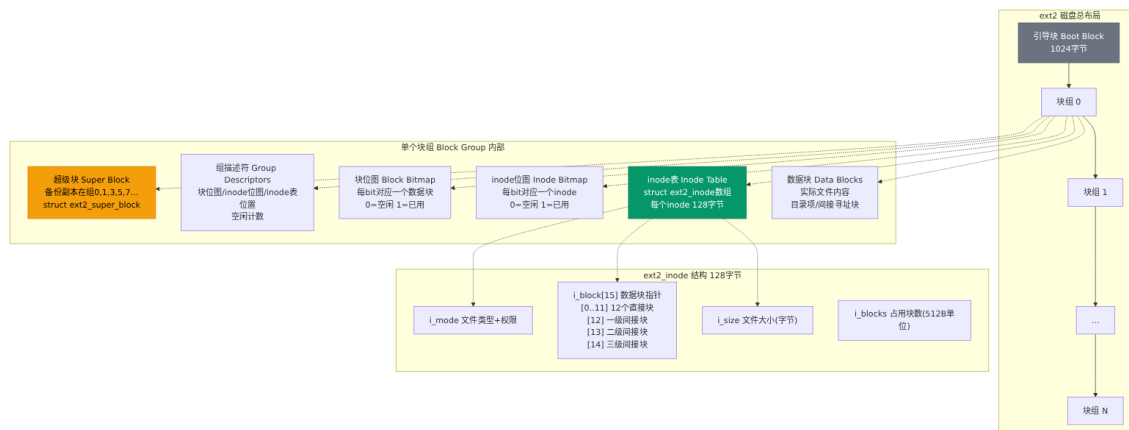


图 12: ext2 文件系统磁盘布局

6.4.1 磁盘布局

ext2 将磁盘划分为等大小的块组 (block group)，每个块组包含：Super Block (文件系统元信息，备份在特定块组中)、Group Descriptors (位图/inode 表位置)、Block Bitmap、Inode Bitmap、Inode Table、Data Blocks。

6.4.2 ext2_inode 与数据块寻址

```
1 struct ext2_inode {
2     __le16 i_mode;           // 文件类型 + 权限
3     __le32 i_size;           // 文件大小 (字节)
4     __le32 i_blocks;         // 占用块数 (512 字节单位)
5     __le32 i_block[15];      // 数据块指针数组
6 };
```

Listing 41: ext2_inode 结构 (fs/ext2/ext2.h)

i_block[15] 是 ext2 数据块寻址的核心，其设计体现了空间效率与寻址能力的经典权衡。对于小文件 (48KB, 假设 4KB 块大小)，仅需 12 个直接块即可完整描述，无需间接块开销。对于中型文件，一级间接块 i_block[12] 指向一个包含 block_size/4 个 32 位指针的数据块 (4KB 块可存 1024 个指针)，额外寻址 1024 块 = 4MB 数据。二级间接 i_block[13] 增加一层间接 (1024×1024=1M 块 = 4GB)，三级间接 i_block[14] 再增一层 (1024³ 块 = 4TB)，足以覆盖 32 位文件偏移上限。

ext2_get_block() (fs/ext2/inode.c) 是这一寻址机制的实现核心。给定文件的逻辑块号 (文件内偏移除以块大小)，该函数递归遍历间接块层级：若逻辑块号落在直接块范围 (< 12)，直接从 i_block[n] 读取物理块号；若落在一级间接范围，先通过 i_block[12] 读取一级间接块，再从中索引；以此类推。若对应物理块尚不存在 (写操作导致文件扩展)，则在各级间接块上分配新块，构建完整路径。

数据块分配策略受 ext2 的块分配算法影响：ext2_new_block() 在目标块组中通过块位图查找空闲块，优先选择与文件已有块物理相邻的块 (减少磁盘寻道)，并预留预分配块供后续扩展使用。

6.4.3 ext2 文件操作

ext2 的 file_operations 定义于 fs/ext2/file.c:

```
1 const struct file_operations ext2_file_operations = {
2     .llseek      = generic_file_llseek,
3     .read_iter   = ext2_file_read_iter,
4     .write_iter  = ext2_file_write_iter,
5     .mmap        = generic_file_mmap,
6     .open        = dquot_file_open,
7     .fsync       = ext2_fsync,
8     .splice_read = ext2_splice_read,
9 };
10
11 static ssize_t ext2_file_read_iter(struct kiocb *iocb, struct iov_iter *to) {
12     // 若不使用 DIO 且为缓冲 I/O, 直接走 page cache 路径
13     return generic_file_read_iter(iocb, to);
14 }
```

Listing 42: ext2_file_operations 与 read_iter/write_iter (fs/ext2/file.c)

generic_file_read_iter() (mm/filemap.c) 实现缓冲 I/O 的标准数据流，是理解文件 I/O 与内存管理协同工作的关键：首先在 page cache 中查找文件偏移对应的 page (filemap_get_folio())

——若命中（大多数顺序读场景），直接通过 `copy_page_to_iter()` 将内核页面的数据复制到用户空间 `iov_iter`；若未命中（首次访问或缓存被回收），分配新 page 并调用 `filemap_read_folio()` → `ext2_read_folio()` 触发磁盘 I/O，将文件数据块读入 page cache，然后复制给用户。

统一 Page Cache——Linux 消除双重缓存的关键设计。经典 Unix 有一个著名的架构问题：文件数据在“buffer cache”（块设备缓存，以磁盘块为单位）和“page cache”（文件页缓存，以页为单位）中各存一份。数据从磁盘读入 buffer cache，再复制到 page cache，再复制到用户空间——两次内存拷贝、两份内存占用、一致性维护复杂。Linux 通过 `struct address_space` 彻底解决了这一问题。

每个文件的 `inode` 拥有一个 `address_space (i_mapping)`，其中的 `i_pages (xarray 基数树)` 以文件偏移量为键，映射到 page cache 中的物理页面。关键设计在于：`ext2_file_read_iter()`（文件 read 路径）调用 `generic_file_read_iter()` → `filemap_read()` 在这个 `xarray` 中查找页面；`ext2_filemap_fault()`（文件 mmap 缺页路径）也在同一个 `xarray` 中查找。如果 `read()` 先访问了文件偏移 0-4095，该页面进入 `address_space`；随后 `mmap` 映射的同一文件同一偏移被访问，缺页处理在 `xarray` 中命中——直接映射该物理页到进程页表，无需再次读磁盘。反之亦然。

这意味着：通过 `read()` 和 `mmap()` 访问同一文件的两个进程，在物理内存中共享的是**完全相同的页面**，而不是两份独立的缓存。`open() → read() → close()` 和 `open() → mmap() → 内存访问 → munmap()` 在 page cache 层面完全统一，只是用户态接口不同。这一设计是 Linux 文件 I/O 架构的核心美学——它消除了 Unix 传承数十年的双重缓存问题，同时保证了一致性（不存在 buffer cache 已更新但 page cache 未更新的窗口）。

实验 9（ext2m 加密文件系统）的设计决策正是基于对此数据流的精确理解：XOR 加解密操作插入在 `read_iter/write_iter` 层而非 `read_folio/writepages` 层，确保 page cache 始终存储密文，`read()` 和 `mmap()` 两个路径共享同样的加密语义，不会因为页缓存的一致性而产生双重加密或遗漏加密的问题。

6.5 实验验证

6.5.1 实验 9：ext2m 加密文件系统实现

本实验通过复制 `fs/ext2/` 为 `fs/ext2m/` 并注册为 “ext2m” 新文件系统类型，实现了透明 XOR 加解密功能。核心修改位于 `fs/ext2m/file.c` 的 `ext2_file_read_iter` 和 `ext2_file_write_iter`。

设计关键：加密操作放置在 `read_iter/write_iter` 层而非更底层的 `read_folio/writepages`。若在 `read_folio` 中解密，`readahead` 预读会在 page cache 中存储解密后的明文，`read_iter` 可能再次解密导致双重解密错误。正确做法是 page cache 始终存储密文，仅在 `generic_file_read_iter()` 返回后（page cache → kernel iov）和 `copy_to_iter()` 前（kernel iov → userspace）之间执行解密。同理，写入时在 `copy_from_iter()` 后（userspace → kernel iov）和 `generic_file_write_iter()` 前（kernel iov → page cache）之间执行加密。XOR 公式为 `buf[i] ^= key[(i + block*7) % 16]`，其中 `block = pos » PAGE_SHIFT`，使得同一文件的不同块使用不同的密钥偏移。

验证方法展示了分层理解：创建 100MB ext2 磁盘镜像（`dd + mkfs.ext2`），以 `-t ext2m` 挂载后 `echo` 写入明文、`cat` 读取明文——加解密对用户透明；以 `-t ext2` 重新挂载同一镜像，`xxd` 查看磁盘原始字节为密文，且首字节 `0x07 = 0x48('H') ^ 0x4F(key[0])` 通过数学验算证实 XOR 运算正确。同一磁盘镜像可在 ext2 和 ext2m 之间交叉挂载而无数据丢失，证实 ext2m 的磁盘格式与 ext2 完全兼容（仅运行时加解密）。该实验完整验证了 VFS 层 `file_operations` 覆盖、page cache 数据流、ext2 磁盘布局以及文件系统类型注册的全链路。

6.5.2 实验 7: Linux 设备文件与驱动程序

本实验编写了两个内核模块，验证了 VFS 的 `file_operations` 在字符设备上的应用。

`hello_module` 作为内核模块的最小骨架，通过 `module_init/exit` 注册加载和卸载回调，`printk` 输出验证模块生命周期。

`char_driver` 实现完整的字符设备驱动。设备注册链路：`alloc_chrdev_region()` 动态分配主设备号（避免了静态指定可能的冲突）→ `cdev_init(&my_cdev, &fops)` 将 `file_operations` 绑定到 `cdev` → `cdev_add()` 注册到内核 → `class_create()` + `device_create()` 触发 `udev` 自动创建 `/dev/mychardev` 设备节点（主设备号 242，动态分配）。`file_operations` 实现了七个回调：`.open`（引用计数递增）、`.read`（通过 `for_each_process(task)` 遍历全部内核进程，使用 `rcu_read_lock/rcu_read_unlock` 保护，`scnprintf` 格式化 PID/状态/优先级/PPID/进程名，`copy_to_user` 输出到用户空间）、`.write`（从用户空间接收数据）、`.unlocked_ioctl`（设备控制命令）、`.release`（引用计数递减）。卸载时按创建逆序释放（`device_destroy` → `class_destroy` → `cdev_del` → `unregister_chrdev_region`），防止资源泄漏。

验证过程：`insmod` 加载后 `ls -la /dev/mychardev` 显示 `c rw----- 1 root root 242, 0`；用户态测试程序依次执行 `open/write/read/ioctl/close` 全链路，`read` 返回的进程列表包含所有用户进程和内核线程。`rmmod` 后设备文件自动消失。该实验验证了 VFS 层 `file_operations` 的统一接口设计——`ext2` 文件的 `read_iter` 和字符设备的 `.read` 均通过 `struct file_operations` 回调提供服务，上层 `read()` 系统调用对二者无差别。

7 总结

本报告以 Linux 6.18.15 内核源码为分析对象，对操作系统内核的核心子系统进行了系统性的源码阅读与分析。

进程管理方面：深入分析了 `task_struct` 进程描述符的结构设计、六态进程状态机的变迁规则、`kernel_clone()` → `copy_process()` 的进程创建全流程（`dup_task_struct` → `sched_fork` → `copy_mm` → `copy_thread` → `wake_up_new_task`），以及系统调用从用户态 `syscall` 指令到 `do_syscall_64()` 分发的完整路径。

进程调度方面（本报告重点）：完整剖析了从 `schedule()` → `__schedule()` → `pick_next_task()` → `context_switch()` → `switch_to` → `__switch_to_asm` → `__switch_to()` 的七层调用链。对 CFS/EEVDF 公平调度算法（`__pick_eevdf()` 红黑树选择逻辑、`entity_tick` 时钟抢占、`enqueue/dequeue` 入队出队）、RT/DL 实时调度以及 x86_64 架构下 RSP 切换点、`callee-saved` 寄存器保存/恢复、CR3 地址空间切换（PCID/ASID）、FPU/FS/GS 架构收尾进行了逐寄存器级的详细分析。

进程间通信方面：分析了 System V IPC 三件套（信号量 `semop` 原子操作、消息队列、共享内存零拷贝）的内核实现、信号的 `send`→`pending`→`handle`→`sigreturn` 全生命周期、管道环形缓冲区机制。

内存管理方面：分析了伙伴系统（`free_area[order]` 分配/释放/分裂/合并）、`slub` 分配器、VMA 与 `mmap` 惰性分配、缺页异常四种处理路径（`anonymous/swap/file/COW`）、`do_wp_page()` 的引用计数优化以及 `kswapd` 页面回收与换出机制。

文件系统方面：分析了 VFS 四对象模型（`super_block/inode/dentry/file`）、`link_path_walk()` 路径名解析（`dcache` 快速路径 + `i_op->lookup()` 慢速路径）、`ext2` 磁盘布局（块组/`super block`/`inode table`/`i_block[15]` 间接块寻址）以及 `read_iter/write_iter` 的 `page cache` 数据流。

7.1 跨子系统设计模式

分别阅读六个子系统后可以识别出一些反复出现的设计模式。这些模式不是某个文件或函数的局部选择，而是 Linux 内核在演化中收敛形成的“设计语言”，理解它们有助于在阅读任何内核代码时更快地建立心智模型：

表 8: Linux 内核跨子系统设计模式

模式	调度器	内存管理	文件系统/VFS	IPC
惰性求值	新进程不分配时间片直到被 <code>pick_task</code> 选中; MFQ 的 <code>mfq_entity_init</code> 在首次 <code>enqueue</code> 时才调用	COW 延迟物理页复制; 缺页异常按需分配页框; VMA 惰性映射 (<code>mmap</code> 只建 VMA 不分配页)	<code>dentry</code> 按需创建 (<code>lookup</code> 慢路径); <code>inode</code> 延迟从磁盘读取	共享内存惰性附加 (<code>shmat</code> 只建 VMA, 首次访问才在 <code>shmem_fault</code> 中映射物理页)
快慢路径分离	<code>pick_next_task</code> 快速路径: 全 CFS 任务时直接走 <code>fair</code> ; 慢速路径: <code>for_each_active_class</code> 遍历全部调度类	<code>get_page_from_freelist</code> 快速路径: 伙伴系统直接分配; 慢速路径: <code>__alloc_pages_slowpath</code> → 压缩 → 回收 → OOM	<code>walk_component</code> 快速路径: <code>dcache</code> 哈希命中 O(1); 慢速路径: <code>i_op->lookup()</code> 磁盘 I/O	<code>futex</code> 快速路径: 用户态 CAS 无竞争直接返回; 慢速路径: <code>futex(FUTEX_WAIT)</code> 进内核睡眠
RCU + 引用计数	<code>rq->curr</code> 用 RCU 保护读取, <code>task_struct</code> 的 <code>usage</code> 引用计数控制释放	<code>folio_ref_count</code> 保护页面生命周期; <code>mm_struct</code> 的 <code>mm_users/mm_count</code> 双重计数	<code>dentry</code> 的 <code>d_lockref</code> (混合 spinlock + 引用计数); <code>inode</code> 的 <code>i_count</code>	<code>sem_array->sem_perm.refcount</code> ; 共享内存 <code>shmid_kernel</code> 的引用计数
per-CPU 隔离	<code>struct rq</code> per-CPU; MFQ <code>DEFINE_PER_CPU(mfq_rq);</code> 调度类无需全局锁	per-CPU <code>cache (struct per_cpu_pages)</code> ; <code>slub</code> 的 per-CPU slab 缓存	—	—
分裂返回	<code>schedule()</code> 在 <code>context_switch</code> 后返回时可能在另一个进程中	<code>fork()</code> 父子同时返回但返回值不同 (子 <code>=0</code> , 父 <code>=pid</code>)	—	<code>semop()</code> 阻塞睡眠后返回: 内核在 <code>perform_atomic_semop</code> 失败后 <code>schedule_timeout</code> , 进程被 V 操作唤醒后“看起来”从 <code>semop()</code> 调用处继续

这五种模式贯穿了从进程管理到文件系统的全部六个子系统。”惰性求值”最小化初始开销 (`fork` 廉价 + COW 减拷贝 + `dcache` 减磁盘 I/O), ”快慢路径分离”保证常见情况零开销 (全 CFS 不走遍历 + `dcache` 命中不走磁盘 + `futex` 无竞争不陷入内核), ”RCU + 引用计数”在性能和安全性之间取得平衡 (读端免锁 + 生命周期精确控制), ”per-CPU 隔离”消除全局锁竞争 (每个 CPU 独立运行队列和页缓存), ”分裂返回”将异步语义包装为同步调用接口 (`fork` 一个调用两个返回 + `schedule` 睡眠透明)。

每章末尾附带的课程实验验证 (共 11 个) 从实践层面印证了源码分析的理论认知。这些实验不仅覆盖了内核各子系统的基本操作, 更关键的是——实验 4 的 XSI 信号量、实验 8 的内核死锁检测、实验 11 的 MFQ 调度器实现——直接触及了内核源码的修改和扩展, 将“阅读”上升到了“理解和操作”的层次。

参考文献

- [1] Linux Kernel Community. Linux 6.18.15 kernel source tree[EB/OL]. <https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git>, 2025.
- [2] Linux Kernel Documentation. CFS Scheduler Design[EB/OL]. [Documentation/scheduler/sched-design-CFS.rst](#).

- [3] Linux Kernel Documentation. EEVDF Scheduler[EB/OL]. `Documentation/scheduler/sched-eevdf.rst`.
- [4] Linux Kernel Documentation. Deadline Task Scheduling[EB/OL]. `Documentation/scheduler/sched-deadline.rst`.
- [5] Linux Kernel Documentation. Memory Management[EB/OL]. `Documentation/admin-guide/mm/`, `Documentation/mm/`.
- [6] Linux Kernel Documentation. Overview of the Linux Virtual File System[EB/OL]. `Documentation/filesystems/vfs.rst`.
- [7] Linux Kernel. ext2 文件系统数据结构定义 [EB/OL]. `fs/ext2/ext2.h`, Linux 6.18.15.
- [8] Robert Love. Linux Kernel Development (3rd Edition)[M]. Addison-Wesley Professional, 2010.
- [9] Daniel P. Bovet, Marco Cesati. Understanding the Linux Kernel (3rd Edition)[M]. O'Reilly Media, 2005.
- [10] Jonathan Corbet, Alessandro Rubini, Greg Kroah-Hartman. Linux Device Drivers (3rd Edition) [M]. O'Reilly Media, 2005.
- [11] Andrew S. Tanenbaum, Herbert Bos. Modern Operating Systems (4th Edition)[M]. Pearson, 2014.
- [12] Mel Gorman. Understanding the Linux Virtual Memory Manager[M]. Prentice Hall, 2004.
- [13] Paul E. McKenney. Is Parallel Programming Hard, And, If So, What Can You Do About It? [M]. kernel.org, 2023.
- [14] Maurice J. Bach. The Design of the UNIX Operating System[M]. Prentice-Hall, 1986.
- [15] W. Richard Stevens, Stephen A. Rago. Advanced Programming in the UNIX Environment (3rd Edition)[M]. Addison-Wesley, 2013.
- [16] Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts (10th Edition)[M]. Wiley, 2018.
- [17] Intel Corporation. Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide[EB/OL]. 2025.
- [18] AMD Corporation. AMD64 Architecture Programmer's Manual, Volume 2: System Programming[EB/OL]. 2024.
- [19] System V Application Binary Interface: AMD64 Architecture Processor Supplement[S]. 2023.
- [20] Jonathan Corbet. CFS scheduler overview[EB/OL]. LWN.net, <https://lwn.net/Articles/230574/>, 2007.
- [21] Jonathan Corbet. The EEVDF scheduler[EB/OL]. LWN.net, <https://lwn.net/Articles/925371/>, 2023.

- [22] Paul E. McKenney. What is RCU?[EB/OL]. LWN.net, <https://lwn.net/Articles/262464/>, 2007.
- [23] Jonathan Corbet. Memory management documentation index[EB/OL]. LWN.net, <https://lwn.net/Kernel/Index/>, 2004–2025.
- [24] Jonathan Corbet. A deep dive into context switching[EB/OL]. LWN.net, <https://lwn.net/Articles/542893/>, 2013.
- [25] Jonathan Corbet. Page fault handling in the Linux kernel[EB/OL]. LWN.net, <https://lwn.net/Articles/546612/>, 2013.
- [26] Peter Zijlstra. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Scheduling Policy[C]. Linux Plumbers Conference, 2023.
- [27] Ingo Molnár. Modular Scheduler Core and Completely Fair Scheduler[C]. Linux Kernel Summit, 2007.
- [28] Fernando J. Corbató, et al. An Experimental Time-Sharing System[C]//Proc. of the Spring Joint Computer Conference. ACM, 1962: 335-344.
- [29] Rémy Card, Theodore Ts'o, Stephen Tweedie. Design and Implementation of the Second Extended Filesystem[C]//Proc. of the First Dutch International Symposium on Linux. 1995.
- [30] William Stallings. Operating Systems: Internals and Design Principles (9th Edition)[M]. Pearson, 2017.